

claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_3e301d0b-e59\agent-a1e681a3d6fa9ff00.jsonl`  
- SHA-256: `f75e19a3c3fe7b8961976c08f1b4c4cb3a11356e13a4d256dd62ceb562009073`  
- Source modified: `2026-06-13T18:49:16+00:00`  
- Imported at: `2026-07-08T16:00:00+00:00`  
- project: `wf_3e301d0b-e59`  
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`
```

Transcript

1. user (2026-06-13T18:45:07.545Z)

Enumerate the no-regression risks and the PURE-LOGIC test plan for a new default-OFF FusionSegmenter + Gate A wiring.

Read: tests/test_trajectory_hybrid_equivalence.py, tests/test_rally_gate.py, tests/test_yolo_hybrid.py, tests/test_config.py to learn the fixture/mocking idioms.

Report: every risk that fusion code could change wasb_hybrid/tracknet_hybrid output (it must not, when min_crossings=0 and fusion not selected); the synthetic fixtures that prove Gate A?B composition, point-in-polygon mask, player-count, and each person feature; how to assert default-OFF leaves the trajectory hybrids byte-identical; and what MUST be owner-verified on GPU (real video). Repo root = C:/Users/avidu/Projects/badminton-highlight-indexer (the indexer).

Read the ACTUAL files and quote file:line. Be precise, not generic.

2. assistant (2026-06-13T18:45:09.080Z)

I'll conduct a thorough analysis of the no-regression risks and the test plan for the FusionSegmenter + Gate A wiring. Let me start by exploring the test files to understand the fixture and mocking idioms.

3. user (2026-06-13T18:45:09.285Z)

```
1      """Equivalence contract for the trajectory-hybrid twins.  
2  
3      tracknet_hybrid and wasb_hybrid were copy-paste twins that drifted (the  
4      shot_count parse); both are now thin subclasses of TrajectoryHybridSegmenter.  
5      These tests run BOTH through the same scenarios and assert identical P3/P4  
6      behavior, so any future divergence has to be introduced deliberately, in the  
7      base, for both at once.  
8      """  
9      from unittest.mock import MagicMock, patch  
10  
11     import pytest  
12  
13     from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter  
14     from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter  
15     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter  
16  
17     TWINS = [  
18         (TrackNetHybridSegmenter, "tracknet", "tracknet_hybrid"),  
19         (WasbHybridSegmenter, "wasb", "wasb_hybrid"),  
20     ]  
21  
22  
23     def _window(start=1.0, end=4.0):  
24         return {  
25             "start": start, "end": end, "active_frame_count": 30,
```

```

26         "peak_velocity": 0.4, "mean_velocity": 0.2, "track_id_count": 1,
27         "chunk_index": 0,
28     }
29
30
31     def _runner(windows=None):
32         runner = MagicMock()
33         runner.healthcheck.return_value = (True, "env ok")
34         runner.run_predict.return_value = "out.csv"
35         runner.parse_trajectory_csv.return_value = [MagicMock(visible=True)]
36         runner.trajectory_to_action_windows.return_value = windows if windows is not None
37     else [_window()]
38         return runner
39
40     def _run(cls, provider_segments, db=None):
41         """Run one segmenter class through the standard mocked pipeline."""
42         provider = MagicMock()
43         provider.analyze_video.return_value = (provider_segments, {})
44         db = db or MagicMock()
45         db.add_candidate_segment.return_value = 55
46         db.add_play_segment.return_value = 200
47
48         with patch.object(cls, "_build_runner", return_value=(_runner(), {"weights_path":
49 "w"})), \
50             patch.object(cls, "_probe_fps_width", return_value=(30.0, 1280.0)), \
51             patch("backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
52 return_value=30.0), \
53             patch("backend.pipeline.segmenters.trajectory_hybrid.extract_video_chunk",
54 return_value=True), \
55             patch("backend.pipeline.segmenters.trajectory_hybrid.provider_registry") as
56 preg, \
57             patch("os.environ.get", return_value="dummy_key"):
58             preg.get.return_value = provider
59             seg = cls(db, {"indexing": {}})
60             return db, seg.process_video("v1", "clip.mp4")
61
62 @pytest.mark.parametrize("cls,family,detector", TWINS)
63 def test_detector_identity_flows_into_db_rows(cls, family, detector):
64     db, res = _run(cls, [{"start_time": 0.5, "end_time": 3.0, "shuttle_exchange_count":
65 4}])
66     assert len(res) == 1
67     _, kwargs = db.add_candidate_segment.call_args
68     assert kwargs["detector"] == detector
69     assert kwargs["detection_params"]["detector"] == detector
70     # P4 metadata tags the family + model.
71     assert res[0]["metadata"]["detector"].startswith(f"{family}-hybrid-")
72     db.link_candidate_to_segment.assert_called_once_with(55, 200)
73
74 @pytest.mark.parametrize("cls,family,detector", TWINS)
75 @pytest.mark.parametrize("exchange,expected", [
76     (4, 4), # provider-reported count wins
77     ("7", 7), # numeric strings coerce
78     (None, 3), # explicit null -> duration estimate (2.5s/0.8 -> 3 floor)
79     ("garbage", 3), # unparseable -> same estimate
80 ])
81 def test_shot_count_behavior_is_identical(cls, family, detector, exchange, expected):
82     """The historical drift point: wasb_hybrid relied on int(None) raising into
83 the fallback while tracknet_hybrid short-circuited. One implementation now."""
84     seg_dict = {"start_time": 0.5, "end_time": 3.0, "shuttle_exchange_count": exchange}
85     _, res = _run(cls, [seg_dict])
86     assert res[0]["metadata"]["shot_count"] == expected

```

```

85
86 @pytest.mark.parametrize("cls,family,detector", TWINS)
87 def test_provider_offsets_applied_identically(cls, family, detector):
88     # Window [1,4] + 2.0s default padding -> clip starts at max(0, -1) = 0.0;
89     # provider-relative 0.5 stays 0.5 absolute here.
90     _, res = _run(cls, [{"start_time": 0.5, "end_time": 3.0}])
91     assert res[0]["start_time"] == 0.5
92     assert res[0]["end_time"] == 3.0
93
94
95 def test_base_is_not_registered():
96     from backend.pipeline.segmenters.base import segmenter_registry
97     available = segmenter_registry.list_available()
98     assert "tracknet_hybrid" in available
99     assert "wasb_hybrid" in available
100     # The base engine itself must not appear as a selectable segmenter.
101     assert not any(isinstance(cls, type) and cls is TrajectoryHybridSegmenter
102                    for cls in available.values())
103

```

4. user (2026-06-13T18:45:09.517Z)

```

1     """Shuttle net-crossing rally gate (pure; no WSL/GPU)."""
2     from backend.pipeline.detectors.tracknet_runner import TrajectoryPoint
3     from backend.pipeline.detectors import rally_gate as rg
4
5
6     def P(frame, x, y, visible=True):
7         return TrajectoryPoint(frame=frame, visible=visible, x=x, y=y)
8
9
10    def test_estimate_play_axis_picks_larger_spread():
11        pts = [P(i, 500 + (i % 2), 50 if i % 2 else 350) for i in range(20)] # y varies, x
~const
12        axis, net, span = rg.estimate_play_axis(pts)
13        assert axis == "y"
14        assert span > 100
15
16
17    def test_count_net_crossings_counts_sign_changes():
18        pts = [P(0, 0, 10), P(1, 0, 200), P(2, 0, 10), P(3, 0, 200)] #
below/above/below/above
19        assert rg.count_net_crossings(pts, "y", net=100.0, deadband=5.0) == 3
20
21
22    def test_deadband_ignores_jitter_near_net():
23        pts = [P(0, 0, 101), P(1, 0, 99), P(2, 0, 102), P(3, 0, 98)] # all within deadband
of net
24        assert rg.count_net_crossings(pts, "y", net=100.0, deadband=10.0) == 0
25
26
27    def test_invisible_points_are_skipped():
28        pts = [P(0, 0, 10), P(1, 0, 200, visible=False), P(2, 0, 10)] # invisible middle ->
no flip
29        assert rg.count_net_crossings(pts, "y", net=100.0, deadband=5.0) == 0
30
31
32    def test_gate_kills_single_toss_keeps_multi_crossing_rally():
33        fps = 10.0
34        rally = [P(i, 500, 50 if i % 2 == 0 else 250) for i in range(0, 10)] # alternates
-> many crossings
35        toss = [P(20, 500, 50), P(25, 500, 250)] # one
crossing
36        pts = rally + toss
37        windows = [(0.0, 1.0), (2.0, 3.0)]

```

```

38     gated = rg.gate_windows(pts, windows, fps, min_crossings=2)
39     assert gated[0].kept and gated[0].crossings >= 2
40     assert not gated[1].kept and gated[1].crossings <= 1
41     assert rg.apply_gate(pts, windows, fps, min_crossings=2) == [(0.0, 1.0)]
42

```

5. user (2026-06-13T18:45:09.720Z)

```

1     """Orchestration tests for HybridYoloSegmenter.
2
3     Stage-1 detection now lives in `PersonDetector` (see test_person_detector.py); these
4     tests mock the segmenter's `self.person` cue seams and assert the segmenter's
5     process_video OUTPUT is unchanged ? that is the regression guard for the fusion-P1
6     extraction (the detector moved, behaviour did not). cv2.VideoCapture is patched in the
7     `person_detector` module now (that is where the capture loop runs).
8     """
9     from unittest.mock import MagicMock, patch
10    from backend.pipeline.segmenters.yolo_hybrid import HybridYoloSegmenter
11
12
13    def _make_cap_mock(num_frames=300):
14        """A cv2.VideoCapture mock yielding `num_frames` readable frames at 30fps."""
15        import cv2
16        cap_mock = MagicMock()
17        cap_mock.isOpened.return_value = True
18
19        def mock_get_prop(prop):
20            if prop == cv2.CAP_PROP_FPS:
21                return 30.0
22            if prop == cv2.CAP_PROP_FRAME_WIDTH:
23                return 1000.0
24            return 0.0
25
26        cap_mock.get.side_effect = mock_get_prop
27        cap_mock.read.side_effect = [(True, "frame")] * num_frames + [(False, None)]
28        return cap_mock
29
30
31    def _make_detections(num_frames=300, step=5, track_id=1):
32        """Per-frame return values for a mocked detect_and_track: one steadily-moving
33        person. Each entry is the list of (track_id, bbox) for that frame, where the box
34        advances `step` px/frame so its normalised velocity clears the test thresholds.
35        """
36        out = []
37        for i in range(num_frames):
38            x = i * step
39            out.append([(track_id, (float(x), 0.0, float(x + 10), 10.0))])
40        return out
41
42
43    def _mock_stage1(segmenter, num_frames=300, step=5):
44        """Wire the segmenter's person-cue detection+tracking to deterministic fakes so
45        tests never need torch model inference, GPU, or a real supervision tracker."""
46        segmenter.person.model = MagicMock() # short-circuits
47        lazy_load_model
48        segmenter.person.make_tracker = MagicMock(return_value=MagicMock())
49        segmenter.person.detect_and_track =
50        MagicMock(side_effect=_make_detections(num_frames, step))
51
52    @patch("backend.pipeline.detectors.person_detector.cv2.VideoCapture")
53    @patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
54    @patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chunk")
55    @patch("backend.pipeline.segmenters.yolo_hybrid.provider_registry")
56    @patch("os.environ.get")

```

```

56 def test_yolo_hybrid_segmenter(mock_env_get, mock_provider_registry, mock_extract,
57                               mock_get_dur, mock_cap):
58     mock_env_get.return_value = "dummy_api_key"
59     mock_get_dur.return_value = 10.0
60     mock_extract.return_value = True
61
62     # Mock provider returned play segment
63     mock_provider = MagicMock()
64     mock_provider.analyze_video.return_value = ([
65         {"start_time": 0.0, "end_time": 3.0, "shuttle_exchange_count": 2}
66     ], {})
67     mock_provider_registry.get.return_value = mock_provider
68
69     db_mock = MagicMock()
70     db_mock.add_play_segment.return_value = "yolo_segment_1"
71
72     config = {
73         "indexing": {
74             "use_gpu": False,
75             "chunk_duration": 100,
76             "chunk_overlap": 10,
77             "yolo_velocity_threshold": 0.1,
78             "dead_time_merge_gap": 2.0,
79             # Process every frame so all 300 frames are tracked
80             "yolo_frame_skip": 1,
81             "min_action_window_duration": 1.0,
82         }
83     }
84
85     segmenter = HybridYoloSegmenter(db_mock, config)
86     _mock_stage1(segmenter)
87     mock_cap.return_value = _make_cap_mock()
88
89     res = segmenter.process_video("v1", "dummy.mp4")
90
91     assert len(res) == 1
92     assert res[0]["id"] == "yolo_segment_1"
93     assert res[0]["start_time"] == 0.0
94     assert res[0]["end_time"] == 3.0
95
96
97     @patch("backend.pipeline.detectors.person_detector.cv2.VideoCapture")
98     @patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
99     @patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chunk")
100    @patch("backend.pipeline.segmenters.yolo_hybrid.provider_registry")
101    @patch("os.environ.get")
102    def test_yolo_hybrid_no_api_key(mock_env_get, mock_provider_registry, mock_extract,
103                                  mock_get_dur, mock_cap):
104        """Segmenter should return empty when GEMINI_API_KEY is missing."""
105        mock_env_get.return_value = None
106
107        db_mock = MagicMock()
108        config = {"indexing": {"use_gpu": False}}
109
110        segmenter = HybridYoloSegmenter(db_mock, config)
111        segmenter.person.model = MagicMock() # short-circuit lazy model load
112
113        res = segmenter.process_video("v1", "dummy.mp4")
114        assert res == []
115
116
117    @patch("backend.pipeline.detectors.person_detector.cv2.VideoCapture")
118    @patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
119    @patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chunk")
120    @patch("backend.pipeline.segmenters.yolo_hybrid.provider_registry")

```

```

121     @patch("os.environ.get")
122     def test_yolo_hybrid_drops_bad_timestamps(mock_env_get, mock_provider_registry,
mock_extract,
123                                     mock_get_dur, mock_cap):
124         """Play segments with unparseable timestamps should be dropped, not silently
kept."""
125         mock_env_get.return_value = "dummy_api_key"
126         mock_get_dur.return_value = 10.0
127         mock_extract.return_value = True
128
129         # Provider returns one good segment and one with garbage timestamps
130         mock_provider = MagicMock()
131         mock_provider.analyze_video.return_value = ([
132             {"start_time": 1.0, "end_time": 3.0},
133             {"start_time": "N/A", "end_time": "??"},
134         ], {})
135         mock_provider_registry.get.return_value = mock_provider
136
137         db_mock = MagicMock()
138         db_mock.add_play_segment.return_value = "segment_ok"
139
140         config = {
141             "indexing": {
142                 "use_gpu": False,
143                 "chunk_duration": 100,
144                 "chunk_overlap": 10,
145                 "yolo_velocity_threshold": 0.05,
146                 "dead_time_merge_gap": 2.0,
147                 "yolo_frame_skip": 1,
148                 "min_action_window_duration": 0.5,
149             }
150         }
151
152         segmenter = HybridYoloSegmenter(db_mock, config)
153         _mock_stage1(segmenter)
154         mock_cap.return_value = _make_cap_mock()
155
156         res = segmenter.process_video("v1", "dummy.mp4")
157
158         # Only the valid segment should survive ? the "N/A" one should be dropped
159         assert len(res) == 1
160         assert res[0]["start_time"] == 1.0
161
162
163     @patch("backend.pipeline.detectors.person_detector.cv2.VideoCapture")
164     def test_extract_windows_returns_enriched_dicts(mock_cap):
165         """The person cue's window extraction should return dicts with motion stats."""
166         config = {"indexing": {"min_action_window_duration": 1.0}}
167         segmenter = HybridYoloSegmenter(MagicMock(), config)
168         _mock_stage1(segmenter)
169         mock_cap.return_value = _make_cap_mock()
170
171         windows = segmenter.person.extract_action_windows_from_chunk(
172             "chunk.mp4", chunk_start=5.0, velocity_thresh=0.05, merge_gap=2.0,
173             frame_skip=1, chunk_index=2,
174         )
175
176         assert len(windows) >= 1
177         w = windows[0]
178         assert set(w) >= {"start", "end", "active_frame_count", "peak_velocity",
179                             "mean_velocity", "track_id_count", "chunk_index"}
180         assert w["chunk_index"] == 2
181         assert w["start"] >= 5.0 # offset into full-video coordinates
182         assert w["active_frame_count"] > 0
183         assert w["peak_velocity"] >= w["mean_velocity"] > 0

```

```

184         assert w["track_id_count"] == 1
185
186
187     @patch("backend.pipeline.detectors.person_detector.cv2.VideoCapture")
188     @patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
189     @patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chunk")
190     @patch("backend.pipeline.segmenters.yolo_hybrid.provider_registry")
191     @patch("os.environ.get")
192     def test_yolo_hybrid_persists_and_links_candidates(mock_env_get, mock_provider_registry,
193                                                       mock_extract, mock_get_dur,
194                                                       mock_cap):
195         """Raw candidates should be stored, and confirmed ones linked to their segment."""
196         mock_env_get.return_value = "dummy_api_key"
197         mock_get_dur.return_value = 10.0
198         mock_extract.return_value = True
199
200         mock_provider = MagicMock()
201         mock_provider.analyze_video.return_value = ([
202             {"start_time": 0.0, "end_time": 3.0, "shuttle_exchange_count": 2}
203         ], {})
204         mock_provider_registry.get.return_value = mock_provider
205
206         db_mock = MagicMock()
207         db_mock.add_candidate_segment.return_value = 77
208         db_mock.add_play_segment.return_value = 101
209
210         config = {
211             "indexing": {
212                 "use_gpu": False,
213                 "chunk_duration": 100,
214                 "chunk_overlap": 10,
215                 "yolo_velocity_threshold": 0.05,
216                 "dead_time_merge_gap": 2.0,
217                 "yolo_frame_skip": 1,
218                 "min_action_window_duration": 1.0,
219             }
220         }
221
222         segmenter = HybridYoloSegmenter(db_mock, config)
223         _mock_stage1(segmenter)
224         mock_cap.return_value = _make_cap_mock()
225
226         res = segmenter.process_video("v1", "dummy.mp4")
227
228         assert len(res) == 1
229         # Candidate persisted with the right detector + reproducibility params
230         assert db_mock.add_candidate_segment.called
231         _, kwargs = db_mock.add_candidate_segment.call_args
232         assert kwargs["detector"] == "yolo_hybrid"
233         assert "peak_velocity" in kwargs["stats"]
234         assert kwargs["detection_params"]["frame_skip"] == 1
235         # Confirmed segment linked back to its candidate
236         db_mock.link_candidate_to_segment.assert_called_once_with(77, 101)
237
238     @patch("backend.pipeline.detectors.person_detector.cv2.VideoCapture")
239     @patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
240     @patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chunk")
241     @patch("backend.pipeline.segmenters.yolo_hybrid.provider_registry")
242     @patch("os.environ.get")
243     def test_yolo_hybrid_store_candidates_disabled(mock_env_get, mock_provider_registry,
244                                                   mock_extract, mock_get_dur, mock_cap):
245         """store_yolo_candidates=False should skip DB persistence (and linking)."""
246         mock_env_get.return_value = "dummy_api_key"
247         mock_get_dur.return_value = 10.0

```

```

248     mock_extract.return_value = True
249
250     mock_provider = MagicMock()
251     mock_provider.analyze_video.return_value = ([
252         {"start_time": 0.0, "end_time": 3.0}
253     ], {})
254     mock_provider_registry.get.return_value = mock_provider
255
256     db_mock = MagicMock()
257     db_mock.add_play_segment.return_value = 101
258
259     config = {
260         "indexing": {
261             "use_gpu": False,
262             "chunk_duration": 100,
263             "chunk_overlap": 10,
264             "yolo_velocity_threshold": 0.05,
265             "dead_time_merge_gap": 2.0,
266             "yolo_frame_skip": 1,
267             "min_action_window_duration": 1.0,
268             "store_yolo_candidates": False,
269         }
270     }
271
272     segmenter = HybridYoloSegmenter(db_mock, config)
273     _mock_stage1(segmenter)
274     mock_cap.return_value = _make_cap_mock()
275
276     segmenter.process_video("v1", "dummy.mp4")
277
278     db_mock.add_candidate_segment.assert_not_called()
279     db_mock.link_candidate_to_segment.assert_not_called()
280

```

6. user (2026-06-13T18:45:09.933Z)

```

1     """Basic tests for the typed configuration package.
2
3     These ensure:
4     - Defaults are sensible and come from the models.
5     - File overrides work.
6     - Old config files with extra keys are tolerated.
7     - save_default_config produces loadable output.
8     """
9
10    import json
11    import tempfile
12    import os
13
14
15    from backend.config import load_config, save_default_config, AppConfig
16
17
18    def test_load_default_config():
19        # Force load without the project's config.json so we test the model built-in
20        defaults
21        cfg = load_config("/nonexistent/path/that/does/not/exist/12345/config.json")
22        assert isinstance(cfg, AppConfig)
23        assert cfg.sport == "badminton"
24        assert cfg.indexing.default_segmenter == "yolo_hybrid"
25        assert cfg.validation.min_blur_threshold == 20.0 # updated default from review
26        assert cfg.indexing.ai_max_workers == 5
27
28    def test_file_override():

```



```

29     with tempfile.TemporaryDirectory() as td:
30         p = os.path.join(td, "override.json")
31         data = {
32             "indexing": {
33                 "default_segmenter": "gemini",
34                 "ai_max_workers": 3,
35             },
36             "validation": {
37                 "min_blur_threshold": 42.0,
38             },
39         }
40         with open(p, "w") as f:
41             json.dump(data, f)
42
43         cfg = load_config(p)
44         assert cfg.indexing.default_segmenter == "gemini"
45         assert cfg.indexing.ai_max_workers == 3
46         assert cfg.validation.min_blur_threshold == 42.0
47         # Unspecified keys fall back to defaults
48         assert cfg.sport == "badminton"
49
50
51 def test_extra_keys_tolerated():
52     """Old or future keys in config.json must not break loading."""
53     with tempfile.TemporaryDirectory() as td:
54         p = os.path.join(td, "with_extra.json")
55         data = {
56             "sport": "badminton",
57             "indexing": {"default_segmenter": "yolo_hybrid"},
58             "some_future_key": "ignored_for_now",
59             "validation": {"relevance": {"court_pixels_min_ratio": 0.1}},
60         }
61         with open(p, "w") as f:
62             json.dump(data, f)
63
64         cfg = load_config(p)
65         assert cfg.indexing.default_segmenter == "yolo_hybrid"
66         assert cfg.validation.relevance.court_pixels_min_ratio == 0.1
67
68
69 def test_unknown_keys_warn_with_dotted_paths(tmp_path, caplog):
70     """Typo'd keys must be LOUD: top-level extras are kept-but-unread and unknown
71     section keys are silently dropped, so without this warning a typo just makes
72     the knob a no-op (the audit's 'silently dropped config' class)."""
73     p = tmp_path / "typo.json"
74     p.write_text(json.dumps({
75         "some_future_key": 1,                # top-level extra
76         "indexing": {"velocity_threshold": 0.5},    # typo'd section key
77         "validation": {"relevance": {"unknown_gate": True}},    # doubly nested
78     })))
79     import logging
80     with caplog.at_level(logging.WARNING, logger="backend.config.loader"):
81         cfg = load_config(str(p))
82         assert isinstance(cfg, AppConfig) # still non-fatal
83         joined = " ".join(r.message for r in caplog.records)
84         assert "some_future_key" in joined
85         assert "indexing.velocity_threshold" in joined
86         assert "validation.relevance.unknown_gate" in joined
87
88
89 def test_recognized_config_does_not_warn(tmp_path, caplog):
90     p = tmp_path / "clean.json"
91     p.write_text(json.dumps({
92         "sport": "badminton",
93         "indexing": {"default_segmenter": "gemini", "wasb": {"velocity_threshold":

```

```

0.02}},
94     )))
95     import logging
96     with caplog.at_level(logging.WARNING, logger="backend.config.loader"):
97         load_config(str(p))
98     assert not [r for r in caplog.records if "CONFIG WARNING" in r.message]
99
100
101 def test_save_default_config_roundtrip():
102     with tempfile.TemporaryDirectory() as td:
103         p = os.path.join(td, "saved.json")
104         saved = save_default_config(p)
105         assert str(saved) == p
106         assert os.path.exists(p)
107
108         cfg = load_config(p)
109         assert cfg.indexing.default_segmenter == "yolo_hybrid"
110         assert cfg.validation.min_blur_threshold == 20.0
111
112
113 def test_model_is_the_source_of_truth():
114     """Changing a default in the model should be reflected without touching json."""
115     # This is a documentation test more than anything.
116     cfg = AppConfig()
117     assert cfg.indexing.tracknet.velocity_threshold == 0.02
118
119
120 def test_output_dir_field_default_and_override():
121     """output_dir now lives on OutputConfig (was a write-only runtime hack), so
122     /api/compile and the CLI resolve the same value. Default + file override + the
123     runtime mutation that backend/main.py:main performs for --output-dir."""
124     cfg = AppConfig()
125     assert cfg.output.output_dir == "output" # built-in default
126
127     # The CLI override path: main() assigns args.output_dir onto the typed model.
128     cfg.output.output_dir = "/tmp/custom_out"
129     assert cfg.output.output_dir == "/tmp/custom_out"
130
131     # File override is honored on load.
132     with tempfile.TemporaryDirectory() as td:
133         p = os.path.join(td, "out_override.json")
134         with open(p, "w") as f:
135             json.dump({"output": {"output_dir": "reels", "db_name": "x.db"}}, f)
136         loaded = load_config(p)
137         assert loaded.output.output_dir == "reels"
138         assert loaded.output.db_name == "x.db"
139
140
141 def test_get_returns_dict_for_nested_sections_legacy_compat():
142     """Regression: legacy consumers (validators/segmenters) do
143     ``config.get("validation", {}).get("min_resolution_width", ...)``. The dict-compat
144     .get must return nested SECTIONS as plain dicts (not Pydantic models), or that
145     chained access raises 'X object has no attribute get' on every real run."""
146     cfg = AppConfig()
147     validation = cfg.get("validation", {})
148     assert isinstance(validation, dict)
149     assert validation.get("min_resolution_width") == 1280
150     # nested-of-nested is also dict-accessible
151     assert isinstance(validation.get("relevance"), dict)
152     assert validation["relevance"].get("court_pixels_min_ratio") == 0.05
153     assert cfg.get("indexing", {}).get("default_segmenter") == "yolo_hybrid"
154     # scalar leaves are returned unchanged; a present scalar ignores the default arg;
155     # a missing key falls back to the default
156     assert cfg.get("ai_provider") == "gemini"
157     assert cfg.get("ai_provider", "ignored-default") == "gemini"

```

```
158         assert cfg.get("does_not_exist", "fallback") == "fallback"
159         # leaf keys inside any section resolve via the dynamic section scan (not a hardcoded
160         # list), so a future top-level section's keys would be discoverable too
161         assert cfg.get("min_resolution_width") == 1280 # lives inside `validation`
162         assert cfg.get("begin_padding") == 0.5         # lives inside `stitching`
163
```

7. assistant (2026-06-13T18:45:11.722Z)

Now let me search for the FusionSegmenter and related code to understand the architecture and identify regression risks.

8. user (2026-06-13T18:45:12.259Z)

No files found

9. user (2026-06-13T18:45:12.574Z)

```
backend\pipeline\detectors\rally_gate.py
tests\test_rally_gate.py
```

10. user (2026-06-13T18:45:12.828Z)

```
backend\pipeline\segmenters\__init__.py
backend\pipeline\segmenters\base.py
backend\pipeline\segmenters\gemini.py
backend\pipeline\segmenters\motion.py
backend\pipeline\segmenters\tracknet_hybrid.py
backend\pipeline\segmenters\wasb_hybrid.py
backend\pipeline\segmenters\trajectory_hybrid.py
backend\pipeline\segmenters\yolo_hybrid.py
```

11. assistant (2026-06-13T18:45:14.064Z)

Let me search for references to FusionSegmenter and Gate A to understand the context better.

12. user (2026-06-13T18:45:14.692Z)

```
Found 1 file
docs\MULTI_SIGNAL_FUSION_PLAN.md
```

13. user (2026-06-13T18:45:15.049Z)

```
Found 15 files
docs\NEXT_STEPS.md
docs\MULTI_SIGNAL_FUSION_PLAN.md
docs\CODE_MAP.md
tests\test_experiment.py
docs\EXPERIMENT_HARNESS.md
backend\eval\experiment.py
CLAUDE.md
backend\eval\calibrate_local.py
backend\eval\distill_local.py
backend\eval\eval_partial_labels.py
tests\test_rally_gate.py
backend\pipeline\detectors\rally_gate.py
backend\eval\export_wasb_segments.py
tests\test_calibrate_local.py
tests\test_export_wasb_segments.py
```

14. assistant (2026-06-13T18:45:16.572Z)

Let me read the fusion plan and the rally_gate implementation, then check the trajectory_hybrid

and base segmenter code.

15. user (2026-06-13T18:45:16.794Z)

```
1      # Multi-signal rally fusion ? add a human (person) cue alongside the shuttle
2
3      **Status:** **P1 DONE** (2026-06-13) ? the person cue is extracted to
4      `backend/pipeline/detectors/person_detector.py` (`PersonDetector`), consumed by
5      `yolo_hybrid` with no
6      behavior change. **P2?P4 remain owner-gated.** Phases below are sequenced but each is a
7      separate PR.
8      **Owner decisions captured** in [S9](#9-open-decisions-for-the-owner).
9
10     > **Companion docs:** the 2-layer mental model is
11     [HOW_RALLY_DETECTION_WORKS.md](HOW_RALLY_DETECTION_WORKS.md);
12     > the modular-fusion architecture this builds on is **ADR-007**
13     ([archives/decisions/DECISIONS.md](archives/decisions/DECISIONS.md));
14     > the permissive-dependency rule is **ADR-002 / ADR-005** ; the audio cue (the #1 next
15     cue, already
16     scaffolded) is [archives/research/AUDIO_RALLY_DETECTION_PLAN.md](archives/research/AUDIO_RALLY_DETECTION_PLAN.md).
17     > The vendor labels that gate the learned path come from
18     [GOLDEN_SET_VENDORING.md](GOLDEN_SET_VENDORING.md)
19     > and the labeling spec in [ROORA_LABELING_GUIDELINES.md](ROORA_LABELING_GUIDELINES.md).
20     > **How each cue here is added/tested without production regression** ? A/B, shadow
21     eval, and the
22     > promote-only-on-lift discipline ? is [EXPERIMENT_HARNESS.md](EXPERIMENT_HARNESS.md)
23     (every P1?P4 cue
24     > below lands flag-gated, default OFF, promoted only on measured LOVO lift).
25
26     ---
27
28     ## 1. Context ? why this, why now
29
30     Rally detection today runs **one cue at a time**. Each segmenter independently turns a
31     single signal
32     into rally windows:
33
34     | Segmenter | Signal | File |
35     |---|---|---|
36     | `wasb_hybrid` / `tracknet_hybrid` | shuttle velocity (frozen WASB/TrackNet trajectory)
37     | `backend/pipeline/detectors/tracknet_runner.py` |
38     | `yolo_hybrid` (the current `default_segmenter`) | person velocity (torchvision +
39     ByteTrack) | `backend/pipeline/segmenters/yolo_hybrid.py` |
40     | `motion` | raw pixel motion | `backend/pipeline/segmenters/motion.py` |
41     | `gemini` | a cloud VLM watching the whole clip | `backend/providers/gemini.py` |
42
43     **There is no fusion.** The owner's directive: make the detector **use the shuttle *and*
44     the humans
45     together** ? and be ready to fold in more signals (audio, later pose) ? delivered
46     **MIT / Apache-2.0 / BSD-clean** so we can ramp commercially without a licensing
47     rethink.
48
49     This is **not greenfield** ? it operationalizes work the repo already blessed:
50
51     - **`HOW_RALLY_DETECTION_WORKS.md` open question:** **"Fusion: combine shuttle-motion
52     (WASB) with
53     player-motion for static cams."**
54     - **ADR-007** already designed a **modular "rally layer"** that fuses cues *outside* the
55     frozen WASB
56     model, with **audio as the #1 next cue and person/pose parked as #2**. This plan picks
57     up the parked
58     person cue and generalizes the fusion seam so future cues are "register a producer,"
59     not "re-touch
60     the segmenter."
```

```

44     - ``backend/pipeline/detectors/rally_gate.py` already exists` ? the net-crossing gate,
explicitly
45     labelled in its own docstring as "Gate A in the over-fire fix spectrum (**B = player-
stance state
46     machine, future)"." The person cue in this plan **is that named-but-unbuilt Gate
B.**
47
48     ### The owner's "held-shuttle" false positive ? what's actually going on
49
50     The owner observed a "rally" that was just a player **holding / flicking the shuttle in
hand** between
51     points. This is the *exact* failure `rally_gate.py` was written to kill:
52
53     > "WASB over-detects: when a player flicks/tosses the shuttle back to the opponent for
service after
54     > losing a point, that single trajectory looks like a rally. The discriminator is
structural: a real
55     > rally has the shuttle crossing the net MULTIPLE times; a single service feed crosses
~once.""
56
57     **The structural fix already exists and is validated offline ? but it is NOT wired into
the production
58     runtime path.** `rally_gate.apply_gate(..., min_crossings=2)` is called only from the
**eval drivers**
59     (`distill_local.py`, `eval_partial_labels.py`, `export_wasb_segments.py`,
`calibrate_local.py`); there
60     is **no `min_crossings` knob in `IndexingConfig`** and the live segmenters don't apply
it. So the
61     single cheapest, highest-confidence win in this whole plan is **P2's first step: fold
the existing
62     Gate A into production.** Everything else (the person cue / Gate B, the learned fusion)
is robustness
63     on top of that.
64
65     ### Why humans help ? and the honest risk
66
67     The beachhead videos are **static tripod** (Bangalore academies,
[PMF_MARKET_RESEARCH.md](PMF_MARKET_RESEARCH.md)),
68     so player presence/motion is a usable cue. ADR-001's warning was specifically about
**broadcast pan**
69     cameras, where player-motion fooled the old heuristic ? *that failure mode does not
apply to a fixed
70     court cam.* But we keep **the shuttle as the camera-invariant primary** so the general
bet stays robust
71     on other camera types. Humans then add:
72
73     - **Precision** ? reject shuttle-on-floor / warm-up knock-ups / held-shuttle feeds when
the court
74     isn't occupied by ?2 players in a play posture.
75     - **Recall** (via the learned classifier) ? features that help bridge shuttle
occlusion/blur gaps.
76
77     ---
78
79     ## 2. Core architecture ? one rally-fusion layer, cues are swappable producers
80
81     Formalize ADR-007's diagram. Every cue is a **black box that emits a time-aligned
signal** (per-frame
82     activity and/or per-window features); **fusion happens in *our* layer, never inside a
frozen model.**
83
84     ` ` `
85     frames ?? WASB (frozen, MIT)          ?? shuttle trajectory ???
86     frames ?? person detector (ours)      ?? tracks + boxes ??????
87     audio ?? hit detector (ours)          ?? hit events ???????????? RALLY LAYER (ours) ??

```

```

rallies
88     [frames ?? pose/stance (ours; parked #3) ?? stance ??????? feature + decision fusion
89     ...
90
91     - **Shuttle = primary, camera-invariant.** Windows still **seed** from
92     `TrackNetRunner.trajectory_to_action_windows()` (`tracknet_runner.py:234`). Other cues
*enrich and
93     adjudicate* shuttle-seeded windows; they do **not** replace the shuttle seed. Person
runs only over
94     **shuttle-seeded candidate windows + padding**, not the whole video ? which bounds the
added cost.
95     - **A small `RallyCue` contract.** Each producer yields `(frame_idx ? features)` + per-
window stats.
96     Adding a cue = register a producer; the existing `SegmenterRegistry`
(`segmenters/base.py:35`) is the
97     template for the pattern. Keep the single-cue segmenters registered as fallbacks.
98
99     ---
100
101     ## 3. The two fusion modes (both ? per the owner decision)
102
103     ### 3.1 Decision-level gate (safe baseline, ships first, no labels needed)
104
105     ...
106     rally_active = shuttle_motion ? (?? persons in court zone ? recent person track)
107     ...
108
109     The `? recent-track` clause tolerates 1?2 frame person-detector misses so the AND-gate
can't cause a
110     recall cliff. This composes with the **existing net-crossing Gate A**:
111
112     ...
113     keep window ? net_crossings ? 2                (Gate A ? EXISTS in rally_gate.py, eval-
only today)
114     ? (?? in-court players ? recent track)    (Gate B ? the person cue, NEW)
115     ...
116
117     Gate A kills the service-feed / held-shuttle case structurally; Gate B adds court-
occupancy evidence
118     for the cases a single trajectory can still fool (e.g. a knock-up that does cross the
net). No training
119     data required ? this is the label-free fusion that ships before any golden labels exist.
120
121     ### 3.2 Feature-level (primary, the learned path)
122
123     The cues emit a **small set of aggregated, scale/translation-invariant per-window
features ? NOT raw
124     coordinates.** Raw pixel coordinates would memorize *this* court/camera and fail to
generalize off a
125     still-small labeled set; counts / ratios / zone-membership / two-sidedness transfer.
126
127     **What already feeds the classifier** (`distill_local.py:40`, `FEATURE_NAMES`):
128     `duration, peak_velocity, mean_velocity, active_ratio, net_crossings` ? note
**`net_crossings` is
129     already a feature** (computed via `rally_gate.gate_windows`).
130
131     **Person features to add (~4, deliberately few):**
132
133     | Feature | Meaning | Rally signal |
134     |---|---|---|
135     | `players_in_court` | median count of persistent in-court tracks | ? 2 (singles) / 4
(doubles); 0?1 = dead time |
136     | `two_sided_motion` | are moving players on **both** net-halves? | rally = two-sided;
one person feeding = one-sided |
137     | `mean_player_motion` | aggregate normalized player velocity | rally = sustained;

```

```

resting/standing = low |
138 | `in_court_ratio` | fraction of frames with ?2 in-court players | track stability vs
flicker |
139 | `shuttle_player_colocation` | fraction of frames the shuttle sits inside/adjacent to a
person box (hand region) | **held = high; in-flight = mostly free space** ? the direct held-
shuttle discriminator |
140
141 These join the shuttle features in the candidate `stats` JSON
142 (`database.py::add_candidate_segment`) and the feature vector of the **existing
distilled classifier ?
143 the tiny numpy-only L2 logistic regression in `backend/eval/classifier.py` (ADR-004),
NOT a new net.**
144 It scores each candidate window `P(real rally)` and replaces the Gemini call at runtime.
145
146 **What the model learns:** the **weights** on those few features ? e.g. down-weight a
shuttle-motion
147 window with 0?1 in-court players / one-sided motion / high shuttle-colocation (warm-up,
floor shuttle,
148 held-shuttle feed); up-weight ?2 players, two-sided, sustained motion, shuttle in free
flight.
149
150 **Small-N discipline (non-negotiable):**
151 - Keep to **~4 person features** ? more features on a still-small labeled set overfit;
that's exactly
152 why the model stays a logistic regression, not a net.
153 - Use the **leave-one-video-out (LOVO)** protocol that `distill_local.py` **already
implements**: train
154 on the other videos' candidates, predict the held-out *video*, score vs its labels ?
never a
155 held-out *window* from the same video (windows in one video are correlated; that
flatters the score).
156 - Training labels today are **Gemini silver labels** (distilled offline, ADR-004/007 ?
Gemini is a
157 teacher, never a runtime dependency, never a training-data *source* for owned
weights); **golden
158 labels are the honest measuring stick**, and as the Roora golden corpus grows they can
also become
159 training labels. Until the labeled set is larger, treat the learned weights as
160 **calibration / direction-check, not "a fully general model"*** ? the decision-level
gate (§3.1) is
161 the shippable, label-free fusion in the meantime.
162
163 > *(Gated on golden labels existing ? the same blocker ADR-007 flags for audio. See
164 > [GOLDEN_SET_VENDORING.md](GOLDEN_SET_VENDORING.md).)*
165
166 ---
167
168 ## 4. Shuttle-state: in-flight vs in-hand (the misclassification, mapped to real code)
169
170 The "shuttle moved" seed conflates a shuttle **in flight** with a shuttle **carried in
hand**. The
171 discriminators, and where each lives:
172
173 | Signal | In-flight | Held / static | Status |
174 |---|---|---|---|
175 | `net_crossings` | ? 1 (usually several) | ~0 (held) / ~1 (single feed) | **EXISTS**
(`rally_gate.py`); eval-only ? wire to prod |
176 | `shuttle_player_colocation` | low (free space) | high (near a hand) | **NEW** (needs
the person cue) |
177 | `direction_reversals` | several (racket contacts) | ~0 | derivable from trajectory;
partial overlap with crossings |
178 | `peak shuttle speed` | high | walking speed | derivable from existing velocity |
179
180 So the in-flight-vs-held fix is **mostly Gate A (already built) + the new colocation
feature from the

```

```

181     person cue**. **No new label column is needed** ? the golden labels already start a
rally at the
182     *serve-contact frame* and treat all pre-serve / held-shuttle time as dead time (see
183     [ROORA_LABELING_GUIDELINES.md](ROORA_LABELING_GUIDELINES.md)), so held-shuttle moments
are negatives
184     by construction and are the ground truth that proves the detector wrong.
185
186     ---
187
188     ## 5. Adversarial mitigations (integrating safely)
189
190     - **Court-region mask** ? polygon around the court; drop persons outside it (spectators,
coaches,
191     adjacent courts). Config-supplied per camera; ~1 ms. Source: a manual per-camera
polygon first;
192     the **4 court corners + net-line** that Roora labels once per video (see the labeling
doc) build
193     this mask **and** the net split that `two_sided_motion` / `net_crossings` need.
194     - **Player-count constraint** ? expect **2 (singles) / 4 (doubles)** persistent in-court
tracks;
195     suppress transient/extra detections. The `singles_or_doubles` manifest flag sets the
expectation.
196     - **Temporal association** ? reuse `supervision` **ByteTrack** (already in
`yolo_hybrid`); reject
197     singleton detections; smooth across frames.
198     - **Multi-court rejection** ? proximity to the primary court region (homography
optional, later).
199     - **Domain shift (COCO frontal ? side/overhead court)** ? start **box-only** (robust);
keep the
200     detector swappable. Fine-tuning a person detector is **parked** ? we have no in-house
person labels,
201     and the **paid-LLM/training guardrail + minors-exclusion** apply to any future person-
training data.
202     - **Fusion-degradation guard** ? shuttle stays primary; person is gate + feature, never
the sole
203     trigger; the single-cue segmenters remain registered fallbacks.
204
205     ---
206
207     ## 6. Box vs pose
208
209     **Box-first.** Occupancy + count + two-sided motion + shuttle-colocation is enough for
both the gate
210     and the features, and box detection is the cheap, robust win. **2D pose is parked behind
the same
211     `RallyCue` interface** ? pose (MediaPipe Pose / RTMPose, both Apache-2.0) buys serve-
ready stance
212     gating and shot cues later, at the cost of a second model. This matches ADR-007 parking
pose as the
213     #2/#3 cue.
214
215     ---
216
217     ## 7. Performance
218
219     The person detector is the **existing in-process torchvision model** run at **frame-skip
3?5**
220     (`yolo_frame_skip` default 3) with track interpolation between sampled frames. The
shuttle keeps
221     running in its separate WSL process ? the two are **separate processes**, so a "shared
backbone" isn't
222     free; budget realistically as the existing `yolo_hybrid` cost minus skip savings. Person
runs **only
223     over shuttle-seeded windows + `ai_handoff_padding`**, not the whole video, which bounds
the cost.

```



```

224
225 ---
226
227 ## 8. Licensing (the guardrail table ? all green)
228
229 Every dependency stays MIT / Apache-2.0 / BSD (ADR-002 / ADR-005; `ultralytics` was
dropped for
230 AGPL-3.0).
231
232 | Cue / model | Code | Weights / data | Verdict |
233 |---|---|---|---|
234 | Shuttle: WASB-SBDT, TrackNetV3 | MIT | frozen badminton weights | ? (ADR-001/002) |
235 | Person (default): torchvision Faster R-CNN / RetinaNet / FCOS | BSD/MIT | COCO-
pretrained | ? already in repo (`_DETECTOR_FACTORIES`) |
236 | Person (escalation): YOLOX, RT-DETR / RT-DETRv2 | Apache-2.0 | COCO /
Objects365 | ? |
237 | Pose (parked): MediaPipe Pose, RTMPose / MMPose | Apache-2.0 | COCO-structure
| ? |
238 | Tracking: `supervision` ByteTrack | MIT | n/a | ? already in repo |
239 | Banned | Ultralytics YOLOv5/8/11 (AGPL-3.0) . MonoTrack (Adobe, non-comm)
. YOLO-NAS (Deci, non-comm) | ? | ? |
240
241 COCO caveat to record: COCO annotations are CC BY 4.0 (commercial OK with
attribution*);
242 COCO images are Flickr-ToS (per-image). Pretrained-weight use is clean; record the
attribution.
243
244 ---
245
246 ## 9. Open decisions (for the owner)
247
248 1. Court-region mask source ? manual per-camera polygon (simple, ship first) vs auto
court
249 homography/line detection (later). Recommend manual first, fed by Roora's court-
corner labels.
250 2. Person detector default ? keep torchvision (zero new deps, BSD/MIT) vs adopt
YOLOX/RT-DETR
251 (Apache, faster) now. Recommend torchvision first, escalate only if eval shows a
speed/accuracy gap.
252 3. Pose now or parked ? recommend parked behind the interface (box-first).
253 4. Golden labels ? feature-level fusion (P3) is blocked on golden rally labels;
the source is
254 the VLC `rally-annotator` + the Roora vendor pilot
([GOLDEN_SET_VENDORING.md](GOLDEN_SET_VENDORING.md)).
255
256 ---
257
258 ## 10. Phased execution (owner-gated; ONE PR per slice, off `master`)
259
260 - P0 ? this design doc* +
[ROORA_LABELING_GUIDELINES.md](ROORA_LABELING_GUIDELINES.md), indexed in
261 `docs/README.md`, cross-linked from `HOW_RALLY_DETECTION_WORKS.md` and ADR-007. *(this
deliverable;
262 docs-only, no code.)*
263 - P1 ? extract the shared person cue:* ? DONE (2026-06-13). Lifted the
torchvision detector +
264 ByteTrack + velocity out of `yolo_hybrid` into
`backend/pipeline/detectors/person_detector.py`
265 (`PersonDetector`); `yolo_hybrid` now orchestrates only and delegates to `self.person`
(no behavior
266 change ? regression-safe, proven by the unchanged process_video output tests). New
267 `tests/test_person_detector.py`; suite +1 green; person_detector is type-clean (left
mypy quarantine
268 for the moved code). The fusion segmenter (P2) reuses this same producer over shuttle-
seeded windows.

```

```

269     - **P2 ? wire Gate A into production, then add the court-mask + player-count gate (Gate
B):** add a
270     `min_crossings` knob + court-region polygon + count constraint to `IndexingConfig`; a
271     `FusionSegmenter` (or extend the hybrid segmenter) runs the person cue over shuttle-
seeded windows
272     and applies `Gate A ? Gate B`. Persist person features into the candidate `stats`.
**Wiring Gate A
273     is the immediate false-positive win.**
274     - **P3 ? feature-level (learned) fusion:** add the person features (incl.
`shuttle_player_colocation`)
275     to `distill_local` / `classifier`; recalibrate IoU on the golden set; report **LOVO
lift on held-out
276     videos**. *(Gated on golden labels.)*
277     - **P4 ? additional cues:** wire **audio** (ADR-007's #1, already scaffolded in
278     `backend/pipeline/audio/hit_detector.py`) and later **pose**, through the same
`RallyCue` interface.
279     Each is add-a-producer, not a refactor.
280
281     ---
282
283     ## 11. Verification
284
285     - **Offline, runnable in this dev container (no GPU/torch/cv2):** unit tests for the
fusion gate logic
286     and `RallyCue` aggregation (court-mask filter, player-count constraint, `? recent-
track` tolerance)
287     and the shuttle-state signals (net-crossing count ? extends the existing `rally_gate`
tests ? and
288     person-colocation) with **synthetic trajectories + person boxes**. A key-license
assertion test
289     (no `ultralytics` import; detector factories stay torchvision/Apache).
290     - **Machine-side (owner, GPU/WSL ? the full pipeline can't run in this container):** run
the
291     `FusionSegmenter` on a real static-tripod clip; confirm **held-shuttle / warm-up /
spectator false
292     positives drop** vs shuttle-only (this should reproduce the owner's observed bug and
show it fixed ?
293     start by confirming Gate-A-in-production alone fixes it); **LOVO IoU lift on the
golden set** for the
294     learned path.
295     - **Docs (P0):** links resolve; both docs indexed in `docs/README.md`; this doc cross-
linked from
296     `HOW_RALLY_DETECTION_WORKS.md` (the fusion open-question) and ADR-007.
297

```

16. user (2026-06-13T18:45:17.005Z)

```

1     """"Rally-start gate: suppress between-point false fires using shuttle net-crossings.
2
3     WASB (shuttle-only) over-detects: when a player flicks/tosses the shuttle back to
4     the opponent for service after losing a point, that single trajectory looks like a
5     rally. The discriminator is structural, not duration-based: a real rally has the
6     shuttle crossing the net MULTIPLE times (each shot lands on the other side); a
7     single service feed crosses ~once.
8
9     This module is camera-agnostic and needs NO new model ? it works on the trajectory
10    we already produce (Frame,Visibility,X,Y). It estimates the play axis (the image
11    axis with the larger shuttle spread) and the net position (median shuttle coord on
12    that axis ? the shuttle clusters at the two ends and crosses the net region often,
13    so the median sits near the net), then counts sign changes of (coord - net) across
14    the visible points inside a candidate window, with a deadband to ignore jitter.
15
16    Gate A in the over-fire fix spectrum (B = player-stance state machine, future).
17    Pure functions only ? no I/O, unit-tested; intended as a post-filter on the
18    windows from trajectory_to_action_windows, or to fold into it later.

```

```

19     """
20     from __future__ import annotations
21
22     from dataclasses import dataclass
23     from typing import List, Sequence, Tuple
24
25     Interval = Tuple[float, float]
26
27
28     @dataclass
29     class GatedWindow:
30         start: float
31         end: float
32         crossings: int
33         visible_points: int
34         kept: bool
35
36
37     def _spread(vals: Sequence[float]) -> float:
38         """Robust spread = p95 - p5 (ignores a few outlier detections)."""
39         if len(vals) < 2:
40             return 0.0
41         s = sorted(vals)
42         lo = s[max(0, int(0.05 * (len(s) - 1)))]
43         hi = s[min(len(s) - 1, int(0.95 * (len(s) - 1)))]
44         return hi - lo
45
46
47     def _median(vals: Sequence[float]) -> float:
48         if not vals:
49             return 0.0
50         s = sorted(vals)
51         n = len(s)
52         return s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2])
53
54
55     def estimate_play_axis(points) -> Tuple[str, float, float]:
56         """Return (axis 'x'/'y', net_value, span) from visible points.
57
58         Play axis = the image axis along which the shuttle travels most (larger spread).
59         For a baseline camera that's Y (players top/bottom); for a side camera, X.
60         """
61         xs = [p.x for p in points if p.visible]
62         ys = [p.y for p in points if p.visible]
63         sx, sy = _spread(xs), _spread(ys)
64         if sx >= sy:
65             return "x", _median(xs), sx
66         return "y", _median(ys), sy
67
68
69     def count_net_crossings(window_points, axis: str, net: float, deadband: float) -> int:
70         """Count sign changes of (coord - net) across visible points, ignoring the
71         deadband zone near the net (so jitter at the net doesn't inflate the count)."""
72         crossings = 0
73         last_side = None
74         for p in window_points:
75             if not p.visible:
76                 continue
77             v = (p.x if axis == "x" else p.y) - net
78             if abs(v) < deadband:
79                 continue
80             side = v > 0
81             if last_side is not None and side != last_side:
82                 crossings += 1
83             last_side = side

```

```

84         return crossings
85
86
87     def _points_in(points, fps: float, start: float, end: float):
88         return [p for p in points if start * fps <= p.frame <= end * fps]
89
90
91     def gate_windows(points, windows: List[Interval], fps: float,
92                     min_crossings: int = 2, deadband_frac: float = 0.06) ->
List[GatedWindow]:
93         """Annotate each (start,end) window with its shuttle net-crossing count and a
94         keep decision (crossings >= min_crossings). Deadband is a fraction of the play
95         axis span. Net/axis are estimated once over the whole trajectory (stable)."""
96         axis, net, span = estimate_play_axis(points)
97         deadband = deadband_frac * span
98         out: List[GatedWindow] = []
99         for (s, e) in windows:
100             wp = _points_in(points, fps, s, e)
101             c = count_net_crossings(wp, axis, net, deadband)
102             vis = sum(1 for p in wp if p.visible)
103             out.append(GatedWindow(start=s, end=e, crossings=c, visible_points=vis,
104                                   kept=c >= min_crossings))
105         return out
106
107
108     def apply_gate(points, windows: List[Interval], fps: float,
109                  min_crossings: int = 2, deadband_frac: float = 0.06) -> List[Interval]:
110         """Convenience: return only the windows that pass the crossing gate."""
111         return [(g.start, g.end) for g in
112                 gate_windows(points, windows, fps, min_crossings, deadband_frac) if g.kept]
113

```

17. user (2026-06-13T18:45:17.194Z)

```

1     """Shared engine for trajectory-substrate hybrid segmenters.
2
3     tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4     standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5     now the single home for the whole pipeline shape:
6
7     healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8     -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10    A concrete subclass supplies only its identity and runner wiring:
11    - ``family`` ? config section under ``indexing.*``, output subdir, and
12      the ``<family>-hybrid-<model>`` detector tag in metadata.
13    - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14    - ``name`` / ``description`` properties (the registry contract).
15    - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16      detection_params extras such as weights_path).
17
18    This base is deliberately NOT registered; only concrete subclasses register.
19    """
20    import os
21    import time
22    import logging
23    import concurrent.futures
24    from typing import Any, Dict, List, Optional, Tuple
25
26    import cv2
27
28    from backend.pipeline.segmenters.base import BasePlaySegmenter
29    from backend.utils.video import get_video_duration, extract_video_chunk
30    from backend.sports import sport_registry
31    from backend.providers import provider_registry

```

```

32     from backend.pipeline.detectors.base import DetectorRunner
33
34     logger = logging.getLogger(__name__)
35
36
37     def _fmt_ts(seconds: float) -> str:
38         seconds = max(0, int(seconds))
39         h, rem = divmod(seconds, 3600)
40         m, s = divmod(rem, 60)
41         return f"{h:02d}:{m:02d}:{s:02d}"
42
43
44     class TrajectoryHybridSegmenter(BasePlaySegmenter):
45         """Trajectory-detector hybrid: precise candidate rallies + AI semantic
46         boundaries."""
47
48         #: config section under `indexing` (velocity_threshold lives there), the
49         #: output subdir for trajectories, and the metadata detector-tag prefix.
50         family: str = ""
51         #: human-readable label used in log lines.
52         display_label: str = ""
53
54         def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
55         Any]]:
56             """Return (runner, detector-specific detection_params extras)."""
57             raise NotImplementedError
58
59         @staticmethod
60         def _probe_fps_width(video_path: str) -> tuple:
61             """Return (fps, frame_width) for a video, with sensible fallbacks."""
62             cap = cv2.VideoCapture(video_path)
63             fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
64             width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
65             cap.release()
66             return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
67
68         @staticmethod
69         def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
70             """Provider-reported exchange count, else a duration-based estimate.
71
72             One canonical implementation ? the twins had drifted (wasb relied on
73             ``int(None)`` raising into the fallback; same result, by accident).
74             """
75             shot_count = segment.get("shuttle_exchange_count")
76             if shot_count is None:
77                 return max(3, int(duration / 0.8))
78             try:
79                 return int(shot_count)
80             except (ValueError, TypeError):
81                 return max(3, int(duration / 0.8))
82
83         def process_video(self, video_id: str, video_path: str, # type: ignore[override]
84             player_pool: Optional[List[str]] = None,
85             max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
86             # override-ignore: the ABC declares the Result contract (Success|Failure)
87             # but every live segmenter still returns a bare list ? the documented
88             # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
89             label = self.display_label
90             # --- Sport profile + AI provider wiring (identical across segmenters) ---
91             sport_name = self.config.get("sport", "badminton")
92             try:
93                 sport_profile = sport_registry.get(sport_name)
94             except KeyError as e:
95                 logger.error(str(e))
96             return []

```

```

95
96         idx_cfg = self.config.get("indexing", {})
97         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
98         model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
99
100         api_key_env_var = f"{provider_name.upper()}_API_KEY"
101         api_key = os.environ.get(api_key_env_var)
102         if not api_key:
103             logger.error(
104                 f"{api_key_env_var} environment variable is not set! "
105                 f"To run the {provider_name} handoff, set your API key. "
106                 f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
107             )
108             return []
109         try:
110             provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
111         except KeyError as e:
112             logger.error(str(e))
113             return []
114
115         video_duration = get_video_duration(video_path)
116         if video_duration <= 0:
117             logger.error("Invalid video duration.")
118             return []
119         fps, frame_width = self._probe_fps_width(video_path)
120
121         # --- Runner config + windowing thresholds ---
122         runner, runner_params = self._build_runner(idx_cfg)
123
124         velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
125         merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
126         min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
127         handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
128         ai_max_workers = idx_cfg.get("ai_max_workers", 5)
129         log_candidates = idx_cfg.get("log_yolo_candidates", True)
130         store_candidates = idx_cfg.get("store_yolo_candidates", True)
131
132         detection_params = {
133             "detector": self.name,
134             "velocity_thresh": velocity_thresh,
135             "merge_gap": merge_gap,
136             "min_action_window_duration": min_window_duration,
137             **runner_params,
138         }
139
140         # --- Phase 1: env healthcheck (fail fast with a clear message) ---
141         ok, msg = runner.healthcheck()
142         if not ok:
143             logger.error("%s WSL environment not ready: %s", label, msg)
144             return []
145         logger.info("%s WSL env OK: %s", label, msg)
146
147         # --- Phase 2: trajectory -> candidate rally windows ---
148         # Trajectory detectors process the whole video in one pass (they carry
149         # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
150         t_start = time.time()
151         out_dir = os.path.join("output", self.family)
152         csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
153         if not csv_path:
154             logger.error("%s produced no trajectory; aborting.", label)
155             return []
156

```

```

157         points = runner.parse_trajectory_csv(csv_path)
158         logger.info("Parsed %d trajectory points (%d visible).",
159                     len(points), sum(1 for p in points if p.visible))
160
161         all_candidate_windows = runner.trajectory_to_action_windows(
162             points, fps=fps, frame_width=frame_width, chunk_start=0.0,
163             velocity_thresh=velocity_thresh, merge_gap=merge_gap,
164             min_window_duration=min_window_duration, chunk_index=0,
165         )
166         logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
167                     label, time.time() - t_start, len(all_candidate_windows))
168
169         if log_candidates:
170             for cw in all_candidate_windows:
171                 logger.info(f"[{label}] rally
172                 {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
173                             f"({cw['end'] - cw['start']:.1f}s) |
174                 frames={cw['active_frame_count']} "
175                             f"peak_v={cw['peak_velocity']:.3f}
176                 mean_v={cw['mean_velocity']:.3f}")
177
178         # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
179         candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
180         if store_candidates:
181             for i, cw in enumerate(all_candidate_windows):
182                 stats = {
183                     "peak_velocity": cw["peak_velocity"],
184                     "mean_velocity": cw["mean_velocity"],
185                     "active_frame_count": cw["active_frame_count"],
186                     "track_id_count": cw["track_id_count"],
187                 }
188                 candidate_ids[i] = self.db.add_candidate_segment(
189                     video_id, detector=self.name,
190                     start_time=cw["start"], end_time=cw["end"],
191                     chunk_index=cw["chunk_index"], confidence=min(1.0,
192                     cw["peak_velocity"]),
193                     stats=stats, detection_params=detection_params,
194                 )
195
196         if not all_candidate_windows:
197             return []
198
199         # --- Phase 3: AI provider handoff over padded candidate clips ---
200         handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
201         os.makedirs(handoff_dir, exist_ok=True)
202         logger.info("Phase 3: %s validation on %d candidate windows...",
203                     provider_name, len(all_candidate_windows))
204
205         def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
206             w_start = max(0, window["start"] - handoff_padding)
207             w_end = min(video_duration, window["end"] + handoff_padding)
208             w_dur = w_end - w_start
209             w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
210             if not extract_video_chunk(video_path, w_start, w_dur, w_path,
211             reencode=True):
212                 return []
213             prompt = sport_profile.get_prompt(chunk_duration=w_dur)
214             segments, _ = provider.analyze_video(w_path, prompt, chunk_duration=w_dur,
215                                                 label=f"Handoff {wi+1}")
216
217             valid = []
218             for seg in segments:
219                 try:
220                     seg["start_time"] = float(seg["start_time"]) + w_start
221                     seg["end_time"] = float(seg["end_time"]) + w_start
222                     seg["candidate_id"] = candidate_ids[wi]

```

```

217         valid.append(seg)
218     except (ValueError, TypeError, KeyError) as e:
219         logger.warning("Dropping segment with unparseable timestamps: %s ?
%s", seg, e)
220     try:
221         os.remove(w_path)
222     except OSError:
223         pass
224     return valid
225
226     ai_segments: List[dict] = []
227     with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
228         futures = [executor.submit(process_candidate_window, wi, cw)
229                     for wi, cw in enumerate(all_candidate_windows)]
230         for future in futures:
231             ai_segments.extend(future.result())
232     try:
233         os.rmdir(handoff_dir)
234     except OSError:
235         pass
236     logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
237
238     # --- Phase 4: DB population + candidate linking ---
239     segments_data = []
240     for segment in ai_segments:
241         try:
242             start = float(segment["start_time"])
243             end = float(segment["end_time"])
244             dur = end - start
245             metadata = {
246                 "shot_count": self._shot_count_from(segment, dur),
247                 "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
248                 "detector": f"{self.family}-hybrid-{model_name}",
249             }
250             segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
251             if segment_id:
252                 candidate_id = segment.get("_candidate_id")
253                 if candidate_id is not None:
254                     self.db.link_candidate_to_segment(candidate_id, segment_id)
255                 segments_data.append({
256                     "id": segment_id, "start_time": start, "end_time": end,
257                     "duration": dur, "metadata": metadata,
258                 })
259         except Exception as e:
260             logger.warning("Skipping invalid segment struct: %s", e)
261
262     return segments_data
263

```

18. assistant (2026-06-13T18:45:18.925Z)

Now let me look at the base segmenter to understand the registry contract, and check what tests exist for the config model.

19. user (2026-06-13T18:45:19.565Z)

```

1     from abc import ABC, abstractmethod
2     from typing import List, Dict, Any, Type, Optional
3     from backend.infrastructure.database import Database
4     from backend.results import Result # new standardized contract (Result[Value] ==
Success[Value] | Failure)

```



```

5
6     class BasePlaySegmenter(ABC):
7         def __init__(self, db: Database, config: Dict[str, Any]):
8             self.db = db
9             self.config = config
10
11         @property
12         @abstractmethod
13         def name(self) -> str:
14             """Unique identifier for this segmenter."""
15             pass
16
17         @property
18         @abstractmethod
19         def description(self) -> str:
20             """Human-readable explanation of this segmenter."""
21             pass
22
23         @abstractmethod
24         def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
25             """
26             Processes a video to detect play segments, populates SQLite database,
27             and returns a Result.
28
29             Use isinstance(result, Failure) (or result.is_success()) to distinguish
30             "no segments found" from "something went wrong" (missing key, healthcheck
31             failure, etc.). This is the start of the standardized error/result contract.
32             """
33             pass
34
35     class SegmenterRegistry:
36         def __init__(self):
37             self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}
38
39         def register(self, segmenter_cls: Type[BasePlaySegmenter]):
40             """Registers a new segmenter class under its name property."""
41             try:
42                 temp_inst = segmenter_cls(None, {})
43                 name = temp_inst.name
44             except Exception as e:
45                 raise ValueError(f"Failed to resolve name property from segmenter class
{segmenter_cls.__name__}: {e}") from e
46             self._segmenters[name] = segmenter_cls
47
48         def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
49             return self._segmenters.get(name)
50
51         def list_available(self) -> Dict[str, str]:
52             """Returns a dict of registered segmenter names and their descriptions."""
53             res = {}
54             for name, cls in self._segmenters.items():
55                 try:
56                     temp_inst = cls(None, {})
57                     desc = temp_inst.description
58                 except Exception:
59                     desc = "No description available."
60                 res[name] = desc
61             return res
62
63     segmenter_registry = SegmenterRegistry()
64

```

```
<tool_use_error>Path does not exist: C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config.py. Note: your current working directory is C:\Users\avidu\Projects\badminton-highlight-indexer.</tool_use_error>
```

21. user (2026-06-13T18:45:21.651Z)

No files found

22. user (2026-06-13T18:45:23.548Z)

```
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\loader.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\__init__.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_config.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_config_wasb.py
```

23. user (2026-06-13T18:45:27.129Z)

```
1      """Pydantic models for the application configuration.
2
3      This module defines the canonical, validated, typed configuration for the
4      Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6      The models support graceful migration from old config.json files via extra="allow".
7      """
8
9      from __future__ import annotations
10
11     from typing import Any, List, Literal, Optional
12
13     from pydantic import BaseModel, Field, field_validator, model_validator
14
15     Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18     class ValidationRelevanceConfig(BaseModel):
19         court_green_lower: list[int] = Field(default=[35, 40, 40])
20         court_green_upper: list[int] = Field(default=[85, 255, 255])
21         court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24     class ValidationConfig(BaseModel):
25         min_resolution_width: int = 1280
26         min_resolution_height: int = 720
27         min_fps: float = 29.9
28         min_blur_threshold: float = 20.0
29         max_scene_cuts_per_minute: float = 0.5
30         relevance: ValidationRelevanceConfig =
31         Field(default_factory=ValidationRelevanceConfig)
32
33
34     class TrackNetConfig(BaseModel):
35         wsl_distro: str = "Ubuntu"
36         repo_dir: str = "~/models/TrackNetV4"
37         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
38         conda_env: str = "TrackNetV4"
39         weights_path: str = ""
40         queue_length: int = 5
41         stage_in_wsl: bool = True
42         wsl_stage_dir: str = "~/clips"
43         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
44         velocity_threshold: float = 0.02
45         # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
46         # decoded frame cache) are deleted automatically ? they are pure intermediates,
```

```

46     # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
47     # true only for debugging. On FAILURE they are always kept so a re-run can resume.
48     keep_frames: bool = False
49     # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
50     # 0 = disabled.
51     min_free_gb: float = 0.0
52
53
54     class WasbIndexConfig(BaseModel):
55         """Typed config for the live WASB shuttle substrate (indexing.wasb).
56
57         Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
58         these knobs actually take effect ? previously this whole block was silently dropped
59         because nested config sections defaulted to extra='ignore'.
60         """
61         wsl_distro: str = "Ubuntu"
62         repo_dir: str = "~/models/WASB-SBDT"
63         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
64         conda_env: str = "wasb"
65         weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
66         sport: str = "badminton"
67         wsl_stage_dir: str = "~/clips"
68         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
69         velocity_threshold: float = 0.02
70         # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
71         # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
72         # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
73         keep_frames: bool = False
74         # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
disabled.
75         min_free_gb: float = 0.0
76
77
78     class IndexingConfig(BaseModel):
79         default_segmenter: str = "yolo_hybrid"
80         use_gpu: bool = True
81         motion_threshold: float = 0.08
82         min_segment_duration: float = 3.0
83         dead_time_merge_gap: float = 2.0
84         chunk_duration: float = 90.0
85         chunk_overlap: float = 10.0
86         detector_model: str = "fasterrcnn_resnet50_fpn"
87         detector_score_threshold: float = 0.5
88         yolo_velocity_threshold: float = 0.15
89         yolo_frame_skip: int = 3
90         yolo_tracker: str = "bytetrack.yaml"
91         yolo_prefetch_workers: int = 2
92         min_action_window_duration: float = 2.0
93         log_yolo_candidates: bool = True
94         store_yolo_candidates: bool = True
95         ai_handoff_padding: float = 2.0
96         ai_max_workers: int = 5
97         # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
98         # chunks of a high-bitrate (4K) source blow past the provider upload cap
99         # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
100         # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
101         # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
102         # Matches gemini_refine's --clip-scale-width default.
103         ai_chunk_scale_width: int = 1280
104         # Optional debug knob: trim every video to the first N seconds before processing.
105         debug_duration: Optional[float] = None
106

```

```

107     tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
108     wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
109
110     @field_validator("default_segmenter")
111     @classmethod
112     def segmenter_must_be_registered(cls, v: str) -> str:
113         # Registry check happens at runtime in main/segmenter selection.
114         # Here we just allow any string for forward compatibility.
115         return v
116
117     @model_validator(mode="after")
118     def _chunk_step_must_be_positive(self) -> "IndexingConfig":
119         # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
120         # step makes `while t < duration: t += step` loop forever, growing memory before
any
121         # GPU work. Reject the bad config at load time instead.
122         if self.chunk_overlap >= self.chunk_duration:
123             raise ValueError(
124                 f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
125                 f"({self.chunk_duration}); otherwise chunking never advances."
126             )
127         return self
128
129
130     class OutputConfig(BaseModel):
131         default_highlights_name: str = "highlights.mp4"
132         db_name: str = "sports_indexer.db"
133         # Directory where the DB, highlight reels, and processed clips are written.
134         # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
135         output_dir: str = "output"
136
137
138     class WatermarkConfig(BaseModel):
139         """Branding overlay burned into each highlight clip during the per-clip re-encode
140         (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
141         (under `stitching.watermark` in config.json) to turn it off. The text is fully
142         configurable. The concat stage stays stream-copy (-c copy)."""
143         enabled: bool = True
144         text: str = "Generated by Khelsutra.guru"
145         logo_path: str = "" # optional transparent PNG overlay; "" = no logo
146         font_path: str = "" # optional .ttf; "" = use the fontconfig `font` family
below
147         font: str = "Sans" # fontconfig family used when font_path is empty
148         font_size_ratio: float = Field(default=0.035, gt=0.0, le=0.5) # text height as a
fraction of frame height
149         opacity: float = Field(default=0.85, ge=0.0, le=1.0)
150         box: bool = True # faint backing box behind the text for legibility
151         margin: int = Field(default=24, ge=0) # px inset from the chosen corner
152         position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = "bottom-
right"
153
154
155     class StitchingConfig(BaseModel):
156         begin_padding: float = 0.5
157         end_padding: float = 0.5
158         use_cache: bool = False
159         min_shot_count: int = 1
160         watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
161
162
163     class SecurityConfig(BaseModel):
164         # When set, /api/process video_path must resolve under this directory (hosted/tenant
165         # mode). Default None preserves the single-user local 'any local file' workflow.

```

```

166     allowed_video_root: Optional[str] = None
167
168     # --- Chunked upload (B2, PR-2) ---
169     # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
170     # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
171     # contain upload_dir or /api/process will reject the uploaded paths.
172     upload_dir: str = "output/uploads"
173     # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
174     # free-tier edge proxies cap request bodies there (HOSTING_PLAN $2).
175     max_upload_mb: int = 4096
176     max_part_mb: int = 95
177     allowed_upload_extensions: List[str] = ["mp4", "mov"]
178
179
180 class AppConfig(BaseModel):
181     """Root configuration object.
182
183     All runtime configuration should be accessed via an instance of this class
184     (or the loader) rather than raw dict lookups.
185     """
186
187     sport: Sport = "badminton"
188     ai_provider: str = "gemini"
189     # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
190     # the alias never deprecates underneath us. Eval reports record the run date
191     ai_model: str = "gemini-flash-latest"
192
193     validation: ValidationConfig = Field(default_factory=ValidationConfig)
194     indexing: IndexingConfig = Field(default_factory=IndexingConfig)
195     output: OutputConfig = Field(default_factory=OutputConfig)
196     stitching: StitchingConfig = Field(default_factory=StitchingConfig)
197     security: SecurityConfig = Field(default_factory=SecurityConfig)
198
199     # Allow extra keys during transition so existing config.json files do not break.
200     model_config = {
201         "extra": "allow",
202         "validate_assignment": True,
203     }
204
205     # Temporary migration shim for legacy dict-style access (config.get("section",
206     {}).get(...)).
207     # Will be removed once all call sites (validators, segmenters, etc.) move to typed
208     # attribute access / the typed config. A nested *section* is returned as a plain
209     # via model_dump, so legacy chained access keeps working ? returning the Pydantic
210     # section model instead breaks it (a model has no .get), which bit every validator
211     # real runs. Scalar leaves are returned as-is.
212     def get(self, key: str, default: Any = None) -> Any:
213         _MISSING = object()
214         val: Any = _MISSING
215         if key in type(self).model_fields or hasattr(self, key):
216             val = getattr(self, key, _MISSING)
217         if val is _MISSING:
218             # Legacy convenience: find a leaf key inside any nested section. Sections
219             # discovered dynamically from the model fields, so a future top-level
220             # (e.g. `audio`, `jobs`) is covered automatically without editing a
221             # hardcoded list.
222             for fname in type(self).model_fields:
223                 section = getattr(self, fname, None)
224                 if isinstance(section, BaseModel) and hasattr(section, key):

```

```

223             val = getattr(section, key)
224             break
225         if val is _MISSING:
226             return default
227         if isinstance(val, BaseModel):
228             return val.model_dump(mode="json")
229         return val
230
231     def __getitem__(self, key: str) -> Any:
232         val = self.get(key)
233         if val is None and key not in type(self).model_fields:
234             # Re-raise like a real dict for unknown top-level
235             raise KeyError(key)
236         return val
237
238
239     # Convenience type alias for callers that want to annotate
240     Config = AppConfig
241

```

24. user (2026-06-13T18:45:27.129Z)

```

1     """Configuration package.
2
3     Public API:
4         from backend.config import load_config, AppConfig, Config
5
6     This package is the canonical place for all configuration access.
7     """
8
9     from .loader import load_config, save_default_config
10    from .models import AppConfig, Config, IndexingConfig, StitchingConfig, ValidationConfig
11    # re-export common ones
12
13    __all__ = [
14        "load_config",
15        "save_default_config",
16        "AppConfig",
17        "Config",
18        "IndexingConfig",
19        "StitchingConfig",
20        "ValidationConfig",
21    ]

```

25. assistant (2026-06-13T18:45:28.865Z)

Now let me check what tests exist for person_detector and search for any fusion-related test scaffolding.

26. user (2026-06-13T18:45:29.727Z)

(Bash completed with no output)

27. user (2026-06-13T18:45:31.350Z)

C:\Users\avidu\Projects\badminton-highlight-indexer/tests/test_person_detector.py

28. user (2026-06-13T18:45:32.489Z)

```

1     """Unit tests for the person-detection rally cue
2     (backend.pipeline.detectors.person_detector).
3
4     Pure/offline ? the torchvision model and supervision tracker are mocked, so these run

```

```

4      without GPU/torch weights. They pin the COCO person==1 filter (the YOLO-was-0 gotcha)
and
5      the motion-window extraction that feeds the candidate `stats` + the learned fusion
features.
6      """
7      from unittest.mock import MagicMock, patch
8
9      from backend.pipeline.detectors.person_detector import PersonDetector, _PERSON_CLASS_ID
10
11
12      def _make_cap_mock(num_frames=300):
13          """A cv2.VideoCapture mock yielding `num_frames` readable frames at 30fps."""
14          import cv2
15          cap_mock = MagicMock()
16          cap_mock.isOpened.return_value = True
17
18          def mock_get_prop(prop):
19              if prop == cv2.CAP_PROP_FPS:
20                  return 30.0
21              if prop == cv2.CAP_PROP_FRAME_WIDTH:
22                  return 1000.0
23              return 0.0
24
25          cap_mock.get.side_effect = mock_get_prop
26          cap_mock.read.side_effect = [(True, "frame")] * num_frames + [(False, None)]
27          return cap_mock
28
29
30      def _make_detections(num_frames=300, step=5, track_id=1):
31          """One steadily-moving person per frame (box advances `step` px/frame)."""
32          return [(track_id, (float(i * step), 0.0, float(i * step + 10), 10.0))]
33                  for i in range(num_frames)]
34
35
36      @patch("backend.pipeline.detectors.person_detector.cv2.VideoCapture")
37      def test_extract_action_windows_returns_enriched_dicts(mock_cap):
38          pd = PersonDetector({"indexing": {"min_action_window_duration": 1.0}})
39          pd.model = MagicMock() # short-circuits
lazy_load_model
40          pd.make_tracker = MagicMock(return_value=MagicMock())
41          pd.detect_and_track = MagicMock(side_effect=_make_detections())
42          mock_cap.return_value = _make_cap_mock()
43
44          windows = pd.extract_action_windows_from_chunk(
45              "chunk.mp4", chunk_start=5.0, velocity_thresh=0.05, merge_gap=2.0,
46              frame_skip=1, chunk_index=2,
47          )
48
49          assert len(windows) >= 1
50          w = windows[0]
51          assert set(w) >= {"start", "end", "active_frame_count", "peak_velocity",
52                           "mean_velocity", "track_id_count", "chunk_index"}
53          assert w["chunk_index"] == 2
54          assert w["start"] >= 5.0 # full-video coordinates
55          assert w["active_frame_count"] > 0
56          assert w["peak_velocity"] >= w["mean_velocity"] > 0
57          assert w["track_id_count"] == 1
58
59
60      def test_detect_and_track_filters_persons_and_returns_track_ids():
61          """detect_and_track keeps only confident PERSON detections, feeds them to the
62          tracker, and returns (track_id, bbox) tuples ? exercising the real torchvision
63          output-shape handling and the COCO person==1 filter (the YOLO-was-0 gotcha)."""
64          import numpy as np
65

```

```

66     pd = PersonDetector({"indexing": {}})
67     pd.device = "cpu"
68
69     # Fake torchvision detector output: a person (kept), a low-score person (dropped),
70     # and a racket/other class (dropped).
71     out = {"boxes": MagicMock(), "labels": MagicMock()}
72     out["boxes"].detach.return_value.cpu.return_value.numpy.return_value = np.array(
73         [[0, 0, 10, 10], [5, 5, 15, 15], [20, 20, 30, 30]], dtype=float)
74     out["labels"].detach.return_value.cpu.return_value.numpy.return_value = np.array(
75         [_PERSON_CLASS_ID, _PERSON_CLASS_ID, 38])          # 38 = sports-ball-ish, not
person
76     out["scores"].detach.return_value.cpu.return_value.numpy.return_value = np.array(
77         [0.9, 0.1, 0.99])
78     pd.model = MagicMock(return_value=out)
79
80     # Fake tracker echoes the detections it receives, assigning a single track id.
81     tracker = MagicMock()
82     tracked = MagicMock()
83     tracked.__len__.return_value = 1
84     tracked.tracker_id = [7]
85     tracked.xyxy = np.array([[0, 0, 10, 10]], dtype=float)
86     tracker.update_with_detections.return_value = tracked
87
88     with patch("backend.pipeline.detectors.person_detector.cv2.cvtColor",
return_value=np.zeros((10, 10, 3))), \
89         patch("backend.pipeline.detectors.person_detector.torch") as mock_torch, \
90         patch.dict("sys.modules", {"supervision": MagicMock()}):
91         mock_torch.no_grad.return_value.__enter__ = lambda s: None
92         mock_torch.no_grad.return_value.__exit__ = lambda s, *a: None
93         # torch.from_numpy(...).permute(...).float().div(...).to(...) -> dummy tensor
94         mock_torch.from_numpy.return_value.permute.return_value.float.return_value.div.r
return_value.to.return_value = "tensor"
95         result = pd.detect_and_track("frame", tracker, score_thresh=0.5)
96
97         # Only the high-confidence person survived the score+class filter
98         assert result == [(7, (0.0, 0.0, 10.0, 10.0))]
99         # And exactly that filtered set (1 box) was handed to the tracker
100         tracker.update_with_detections.assert_called_once()
101

```

29. assistant (2026-06-13T18:45:34.162Z)

Now let me examine the experiment harness and the `distill_local` to see the test idiom for feature assembly and LOVO evaluation.

30. user (2026-06-13T18:45:35.030Z)

```

1     """Distill Gemini (teacher) into a LEARNED local rally classifier (student).
2
3     Threshold-tuning the local knobs hit a low ceiling (see calibrate_local). This goes
4     further: learn "is this WASB candidate a real rally?" from Gemini silver labels,
5     using a few resolution/fps-INVARIANT local features, so the model transfers across
6     videos and runs 100% local at inference (no Gemini, privacy lever intact).
7
8     Flow (all local at runtime):
9     WASB trajectory -> recall-oriented candidate windows (propose many; classifier
10     filters) -> per-candidate features [duration, peak_velocity, mean_velocity,
11     active_ratio, net_crossings] -> LogisticRegression(prob >= threshold) -> merge
12     -> rally windows.
13
14     Training labels come from Gemini full-context labels via the SAME IoU matching the
15     evaluator uses (training.label_candidates). Honest leave-one-video-out: train on the
16     other videos' candidates, predict the held-out one, score vs its Gemini labels, and
17     compare against the threshold baseline. Reuses classifier.py, training.label_candidates,
18     rally_gate, metrics, calibrate_wasb.load_golden_gts, gemini_refine.merge_overlaps.

```



```

19
20     Run:
21         python -m backend.eval.distill_local --manifest videos.json --model-out
output/local_rally_model.json
22     """
23     from __future__ import annotations
24
25     import json
26     import argparse
27     from typing import Dict, List, Optional, Tuple
28
29
30     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
31     from backend.pipeline.detectors import rally_gate
32     from backend.eval.calibrate_wasb import load_golden_gts
33     from backend.eval.training import label_candidates
34     from backend.eval.classifier import LogisticRegression
35     from backend.eval.metrics import score
36     from backend.eval.gemini_refine import merge_overlaps
37
38     Interval = Tuple[float, float]
39
40     FEATURE_NAMES = ["duration", "peak_velocity", "mean_velocity", "active_ratio",
"net_crossings"]
41     # Recall-oriented proposal: deliberately permissive so real rallies are among the
42     # candidates (the classifier removes the junk). (velocity_thresh, merge_gap, min_dur)
43     PROPOSAL = (0.005, 1.0, 0.5)
44     BASELINE_LOCAL = (0.02, 2.0, 2.0) # today's production-ish windowing, no learning
45
46
47     def build_candidates(trajecory_csv: str, fps: float, frame_width: float,
48                         proposal: Tuple[float, float, float] = PROPOSAL) ->
Tuple[List[Interval], List[List[float]]]:
49     """WASB trajectory -> (candidate intervals, fps/resolution-invariant feature
rows)."""
50     points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
51     vt, mg, mwd = proposal
52     wins = TrackNetRunner.trajectory_to_action_windows(
53         points, fps=fps, frame_width=frame_width,
54         velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
55     intervals = [(w["start"], w["end"]) for w in wins]
56     gated = rally_gate.gate_windows(points, intervals, fps, min_crossings=0) # for net-
crossings
57     X: List[List[float]] = []
58     for w, g in zip(wins, gated):
59         dur = max(1e-6, w["end"] - w["start"])
60         win_frames = max(1.0, dur * fps)
61         X.append([
62             dur, # rally length (sec) ?
63             float(w["peak_velocity"]), # already normalized by
64             float(w["mean_velocity"]),
65             float(w["active_frame_count"]) / win_frames, # active-shuttle ratio ?
66             float(g.crossings), # net crossings (count) ?
67             the rally signal
68         ])
69     return intervals, X
70
71     def baseline_windows(trajecory_csv: str, fps: float, frame_width: float) ->
List[Interval]:
72     points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
73     vt, mg, mwd = BASELINE_LOCAL

```

```

74     wins = TrackNetRunner.trajectory_to_action_windows(
75         points, fps=fps, frame_width=frame_width,
76         velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
77     return [(w["start"], w["end"]) for w in wins]
78
79
80 def load_video(spec: Dict, iou: float = 0.5) -> Dict:
81     fps, fw = float(spec["fps"]), float(spec["frame_width"])
82     intervals, X = build_candidates(spec["trajectory"], fps, fw)
83     gts = load_golden_gts(spec["labels"])
84     y = label_candidates(intervals, gts, iou)
85     return {"name": spec.get("name", spec["trajectory"]), "intervals": intervals, "X":
X,
86         "y": y, "gts": gts, "baseline": baseline_windows(spec["trajectory"], fps,
fw)}
87
88
89     def predict_rallies(model: LogisticRegression, X: List[List[float]], intervals:
List[Interval],
90         threshold: float = 0.5) -> List[Interval]:
91         if not intervals:
92             return []
93         proba = model.predict_proba(X)
94         pos = [iv for iv, p in zip(intervals, proba) if p >= threshold]
95         return merge_overlaps(pos, gap_tol=1.0)
96
97
98     def run(specs: List[Dict], iou: float = 0.5, threshold: float = 0.5,
99         model_out: Optional[str] = None) -> dict:
100         videos = [load_video(s, iou) for s in specs]

```

31. user (2026-06-13T18:45:35.636Z)

```

40: FEATURE_NAMES = ["duration", "peak_velocity", "mean_velocity", "active_ratio",
"net_crossings"]
56:     gated = rally_gate.gate_windows(points, intervals, fps, min_crossings=0) # for net-
crossings
116:         model = LogisticRegression().fit(X, y, FEATURE_NAMES)
135:     final = LogisticRegression().fit(Xall, yall, FEATURE_NAMES)
137:     weights = dict(zip(FEATURE_NAMES, [round(float(w), 3) for w in final.w]))

```

32. assistant (2026-06-13T18:45:37.262Z)

Now let me look at the person_detector to understand the interface:

33. user (2026-06-13T18:45:37.874Z)

C:\Users\avidu\Projects\badminton-highlight-
indexer/backend/pipeline/detectors/person_detector.py

34. user (2026-06-13T18:45:39.216Z)

```

1     """Person-detection rally cue ? torchvision detector + ByteTrack + motion windowing.
2
3     Extracted verbatim from `yolo_hybrid` (fusion P1, `docs/MULTI_SIGNAL_FUSION_PLAN.md`):
4     the **person cue is a swappable producer**, not logic baked into one segmenter. Today
5     `HybridYoloSegmenter` is the only consumer (it delegates Stage-1 here, byte-
identically);
6     next the fusion segmenter consumes the SAME producer over shuttle-seeded windows, and
7     later the learned classifier reads its persisted features. Keeping detection here (not
in
8     the segmenter) is what makes "add a cue = register a producer" true.
9
10    Behaviour is unchanged from the in-lined version ? only the home moved:

```

```

11     - `lazy_load_model`           ? load a permissively-licensed torchvision detector
12     - `make_tracker`             ? a fresh supervision ByteTrack per chunk
13     - `detect_and_track`         ? one frame ? confident persons with persistent IDs
14     - `extract_action_windows_from_chunk` ? a chunk ? high-motion candidate windows
15       with the motion stats (`peak/mean_velocity`, `active_frame_count`, `track_id_count`)
16       that feed the candidate `stats` JSON and the learned fusion features.
17
18     Heavy deps (torchvision, supervision) are imported lazily inside the methods so
importing
19     the segmenter registry never pulls torch on a machine that lacks it (and tests can mock
20     the seams). Licensing: torchvision detectors are BSD + COCO weights; ByteTrack is MIT
21     (ADR-005; ultralytics' AGPL YOLO was dropped).
22     """
23     import logging
24     from typing import Any, Dict, List, Optional, Tuple
25
26     import cv2
27     import torch
28
29     from backend.utils.geometry import get_velocity_normalised
30
31     logger = logging.getLogger(__name__)
32
33     # torchvision detection models are trained on COCO where the "person" category is
34     # class 1 (index 0 is the implicit background). NB: ultralytics YOLO used class 0 ?
35     # this off-by-one is the classic gotcha when swapping detectors. Keep it named.
36     _PERSON_CLASS_ID = 1
37
38     # Permissively-licensed torchvision detectors selectable via config
39     # `indexing.detector_model`. All are BSD-licensed (code + COCO weights).
40     _DETECTOR_FACTORIES = {
41         "fasterrcnn_resnet50_fpn": "fasterrcnn_resnet50_fpn",
42         "fasterrcnn_mobilenet_v3_large_fpn": "fasterrcnn_mobilenet_v3_large_fpn",
43         "retinanet_resnet50_fpn": "retinanet_resnet50_fpn",
44     }
45     _DEFAULT_DETECTOR = "fasterrcnn_resnet50_fpn"
46
47
48     class PersonDetector:
49         """Stage-1 person detection + tracking + motion-window extraction (a rally cue).
50
51         Owns the torchvision model + per-chunk ByteTrack tracker. Constructed with the run
52         ``config`` (reads ``indexing.{use_gpu, detector_model, detector_score_threshold,
53         min_action_window_duration}``); the model loads lazily on first use.
54         """
55
56         def __init__(self, config: Dict[str, Any]):
57             self.config = config
58             # Dynamic torchvision module (None until lazy_load_model); typed Any so the
59             # `self.model([tensor])` inference call type-checks without a quarantine.
60             self.model: Any = None
61             self.device: Optional[str] = None
62
63         def lazy_load_model(self) -> None:
64             if self.model is not None:
65                 return
66
67             # Imported lazily so the module imports without torchvision present and so
68             # the test suite (which pre-sets self.model) never pulls in model weights.
69             from torchvision.models import detection as tv_detection
70
71             idx_cfg = self.config.get("indexing", {})
72             use_gpu = idx_cfg.get("use_gpu", True)
73             self.device = "cuda" if use_gpu and torch.cuda.is_available() else "cpu"
74             if use_gpu and not torch.cuda.is_available():

```

```

75         logger.warning("use_gpu is True but CUDA is not available. Falling back to
CPU.")
76
77         model_name = idx_cfg.get("detector_model", _DEFAULT_DETECTOR)
78         if model_name not in _DETECTOR_FACTORIES:
79             logger.warning(f"Unknown detector_model '{model_name}'. Falling back to
{_DEFAULT_DETECTOR}.")
80             model_name = _DEFAULT_DETECTOR
81
82         builder = getattr(tv_detection, _DETECTOR_FACTORIES[model_name])
83         logger.info(f"Loading torchvision detector '{model_name}' on {self.device}...")
84         # weights="DEFAULT" pulls the best available COCO-pretrained checkpoint.
85         self.model = builder(weights="DEFAULT")
86         self.model.eval()
87         self.model.to(self.device)
88
89     def make_tracker(self, fps: float) -> Any:
90         """Create a fresh ByteTrack tracker for one chunk (track IDs are chunk-local).
91
92         Imported lazily so tests can mock this method without supervision installed.
93         """
94         import supervision as sv
95         frame_rate = max(1, int(round(fps)))
96         return sv.ByteTrack(frame_rate=frame_rate)
97
98     def detect_and_track(self, frame: Any, tracker: Any,
99                         score_thresh: float) -> List[Tuple[int, Tuple[float, float,
float, float]]]:
100         """Detect persons in a frame and assign persistent track IDs.
101
102         Returns a list of (track_id, bbox) where bbox is (x1, y1, x2, y2) in pixels.
103         This replaces the old ultralytics ``model.track(...)`` call, which bundled
104         detection AND association: torchvision does the detection, supervision's
105         ByteTrack does the association.
106         """
107         import supervision as sv
108
109         # cv2 frames are BGR uint8 HxWxC; torchvision wants RGB float [0,1] CxHxW.
110         rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
111         tensor = torch.from_numpy(rgb).permute(2, 0,
112         1).float().div(255.0).to(self.device)
113         with torch.no_grad():
114             outputs = self.model([tensor])
115
116         out = outputs[0]
117         boxes = out["boxes"].detach().cpu().numpy()
118         labels = out["labels"].detach().cpu().numpy()
119         scores = out["scores"].detach().cpu().numpy()
120
121         # Keep only confident person detections.
122         mask = (labels == _PERSON_CLASS_ID) & (scores >= score_thresh)
123         if not mask.any():
124             return []
125
126         # ByteTrack requires confidence to be populated (it filters on it internally).
127         dets = sv.Detections(xyxy=boxes[mask], confidence=scores[mask],
class_id=labels[mask])
128         tracked = tracker.update_with_detections(dets)
129
130         result: List[Tuple[int, Tuple[float, float, float, float]]] = []
131         for i in range(len(tracked)):
132             tid = tracked.tracker_id[i]
133             if tid is None:
134                 continue
135             x1, y1, x2, y2 = tracked.xyxy[i]

```

```

135         result.append((int(tid), (float(x1), float(y1), float(x2), float(y2))))
136     return result
137
138     def extract_action_windows_from_chunk(self, chunk_path: str, chunk_start: float,
139                                         velocity_thresh: float, merge_gap: float,
140                                         frame_skip: int = 3,
141                                         chunk_index: Optional[int] = None) ->
List[Dict[str, Any]]:
142         """
143         Runs person detection + tracking on a video chunk and extracts high-motion
144         action windows. Returns a list of dicts in the full video's timeframe, each
with:
145             start, end (seconds), active_frame_count, peak_velocity, mean_velocity,
146             track_id_count, chunk_index.
147
148         Args:
149             frame_skip: Process every Nth frame (1 = every frame). Higher values
150                 dramatically reduce inference cost with minimal quality loss.

```

35. user (2026-06-13T18:45:41.019Z)

```

150                 dramatically reduce inference cost with minimal quality loss.
151             chunk_index: Index of the source chunk (recorded on each window for
traceability).
152         """
153         cap = cv2.VideoCapture(chunk_path)
154         if not cap.isOpened():
155             logger.error(f"Could not open {chunk_path}")
156             return []
157
158         fps = cap.get(cv2.CAP_PROP_FPS)
159         if fps <= 0:
160             fps = 30.0
161
162         frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
163         if frame_width <= 0:
164             frame_width = 1280.0 # Sensible fallback
165
166         # Effective sampling rate after frame-skipping
167         effective_fps = fps / max(frame_skip, 1)
168
169         # A fresh tracker per chunk ? track IDs only need to be consistent within a
170         # chunk (windows are computed per chunk, then merged across chunks by time).
171         tracker = self.make_tracker(effective_fps)
172         score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
0.5)
173
174         # Per active frame we keep (frame_idx, peak_velocity, set_of_active_track_ids)
175         # so windows can be enriched with motion stats for logging + fine-tuning.
176         active_frames = []
177         frame_idx = 0
178         track_history: Dict[int, Tuple[float, float, float, float]] = {}
179
180         while True:
181             ret, frame = cap.read()
182             if not ret:
183                 break
184
185             # Skip frames for performance ? at 30fps, every 3rd frame still
186             # gives ~10 samples/second, plenty for human-scale velocity.
187             if frame_idx % max(frame_skip, 1) != 0:
188                 frame_idx += 1
189                 continue
190
191             # Detect persons + assign persistent track IDs.

```

```

192         detections = self.detect_and_track(frame, tracker, score_thresh)
193
194         frame_peak_velocity = 0.0
195         frame_active_ids = set()
196         current_ids = set()
197
198         for track_id, bbox in detections:
199             current_ids.add(track_id)
200
201             if track_id in track_history:
202                 prev_bbox = track_history[track_id]
203                 velocity = get_velocity_normalised(prev_bbox, bbox, effective_fps,
frame_width)
204                 if velocity > velocity_thresh:
205                     frame_peak_velocity = max(frame_peak_velocity, velocity)
206                     frame_active_ids.add(track_id)
207
208                 track_history[track_id] = bbox
209
210             # Prune stale IDs that are no longer visible ? prevents false
211             # velocity spikes when the tracker reassigns an old ID.
212             track_history = {k: v for k, v in track_history.items() if k in current_ids}
213
214             if frame_active_ids:
215                 active_frames.append((frame_idx, frame_peak_velocity, frame_active_ids))
216
217             frame_idx += 1
218
219     cap.release()
220
221     # Group active frames into continuous windows
222     if not active_frames:
223         return []
224
225     max_gap_frames = int(merge_gap * fps)
226
227     def _finalize(group: List[Tuple[int, float, set]]) -> Dict[str, Any]:
228         velocities = [v for _, v, _ in group]
229         track_ids = set().union(*[ids for _, _, ids in group])
230         return {
231             "start": group[0][0] / fps + chunk_start,
232             "end": group[-1][0] / fps + chunk_start,
233             "active_frame_count": len(group),
234             "peak_velocity": max(velocities),
235             "mean_velocity": sum(velocities) / len(velocities),
236             "track_id_count": len(track_ids),
237             "chunk_index": chunk_index,
238         }
239
240     windows = []
241     group = [active_frames[0]]
242     for af in active_frames[1:]:
243         if af[0] - group[-1][0] <= max_gap_frames:
244             group.append(af)
245         else:
246             windows.append(_finalize(group))
247             group = [af]
248     windows.append(_finalize(group))
249
250     # Filter out extremely short action windows
251     min_window_duration = self.config.get("indexing",
}).get("min_action_window_duration", 2.0)
252     windows = [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
253
254     return windows

```

36. assistant (2026-06-13T18:45:43.398Z)

Perfect. Now let me check the tracking logic in the code to understand the current hybrid implementation, and look for any existing experiment infrastructure:

37. user (2026-06-13T18:45:44.144Z)

(Bash completed with no output)

38. assistant (2026-06-13T18:45:45.482Z)

Now let me check if there's already a FusionSegmenter stub or if it needs to be created:

39. user (2026-06-13T18:45:45.853Z)

```
ls: cannot access 'C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\*.py': No such file or directory
```

40. user (2026-06-13T18:45:47.471Z)

```
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\base.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\motion.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\tracknet_hybrid.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\trajectory_hybrid.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\wasb_hybrid.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\yolo_hybrid.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\__init__.py
```

41. user (2026-06-13T18:45:48.805Z)

```
1 from backend.pipeline.segmenters.base import segmenter_registry, BasePlaySegmenter
2 from backend.pipeline.segmenters.motion import MotionPlaySegmenter
3 from backend.pipeline.segmenters.gemini import GeminiPlaySegmenter
4 from backend.pipeline.segmenters.yolo_hybrid import HybridYoloSegmenter
5 from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
6 from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
7
8 __all__ = [
9     "segmenter_registry",
10    "BasePlaySegmenter",
11    "MotionPlaySegmenter",
12    "GeminiPlaySegmenter",
13    "HybridYoloSegmenter",
14    "TrackNetHybridSegmenter",
15    "WasbHybridSegmenter",
16 ]
17
```

42. assistant (2026-06-13T18:45:50.210Z)

Now let me check the CLAUDE.md file to see if there's context about the fusion work:

43. user (2026-06-13T18:45:50.271Z)

```
1 # CLAUDE.md ? project memory / agent handoff
2
3 Local, AI-powered **badminton (? racket-sports) highlight indexer + rally detector**:
4 FastAPI + SQLite + ffmpeg + GPU CV/AI, with paid **Gemini** as the cloud AI. Sibling
```

```

5      repos: `sports-data-collector`, a VLC-based `rally-annotator`.
6
7      ## Read these first
8      - **`docs/README.md`** ? doc index. **`docs/CODE_MAP.md`** ? code navigation + data
9        contracts (the cold-start map; read before touching code).
10     - **Authoritative owned-model roadmap:** `docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md` (+
the
11       live `docs/NEXT_STEPS.md`). These SUPERSEDE the 2026-06-07 strategy docs below where
they
12       disagree (licensing, Gemma-4 stance, base model, conversational-QA).
13     - **Strategy (post-scaffolding):** `docs/COMMERCIALIZATION.md`,
`docs/PMF_MARKET_RESEARCH.md`,
14       `docs/PLATFORM_ARCHITECTURE.md`, `docs/DEFERRED_HARDENING_PLAN.md`. Also
`docs/ROADMAP.md`
15       (offline-segmenter narrative) and `docs/BACKLOG.md`.
16     - **Rally-detection quality track (design merged 2026-06-13, owner-gated, NOT
started):**
17       `docs/HOW_RALLY_DETECTION_WORKS.md` (2-layer mental model) ?
`docs/MULTI_SIGNAL_FUSION_PLAN.md`
18       (fuse shuttle + person + audio cues; the held-shuttle bug is the already-built
`rally_gate.py`
19       "Gate A" **not wired into production** ? cheapest win) ? `docs/EXPERIMENT_HARNESS.md`
(A/B + shadow
20       eval so cues land flag-gated/default-OFF with zero regression) ?
`docs/ROORA_LABELING_GUIDELINES.md`
21       (v8 vendor labeling spec). **`docs/NEXT_STEPS.md` has the prioritized pick-up for this
track.**
22     - **History:** `docs/archives/` ? ADRs (`decisions/DECISIONS.md`), research, and
23       **`checkpoints/`** (latest: `2026-06-strategy-foundation/`). Archive policy: only move
24       **terminal-state** (DONE/REJECTED) work; live docs stay at `docs/` top level.
25
26     ## Current state (June 2026)
27     - **Scaffolding complete** ; strategy foundation + owned-model roadmap merged. **No
28       platform/owned-model implementation has started.** A 2026-06-10 audit-driven hardening
29       sweep landed (licensing gate, detector keying, re-ingest safety, config, eval gate,
API
30       validation, reliability, CI + test coverage) ? see git history.
31     - **Two tracks, both owner-gated:** the **committed next PLATFORM slice = async job
model**
32       (`DEFERRED_HARDENING_PLAN.md` #16, single-tenant); the **owned-model track** starts at
a
33       greenlit Phase-0 milestone (`OWNED_MODEL_IMPLEMENTATION_PLAN.md`). **Do not start
34       implementation without explicit owner approval.**
35
36     ## Architecture in one breath
37     - **Pluggable registries** (ABC + `Registry` singleton): segmenters / providers /
validators
38       / sports / annotations ? see `backend/pipeline/segmenters/base.py`. *Future:*
39       `StorageBackend` + `ComputeBackend` registries (PLATFORM §3).
40     - `video_id` = MD5 of the file ? `backend/utils/hashing.py::compute_video_id` (one
helper).
41     - Typed config = `backend/config/` (Pydantic). DB schema evolves via the `PRAGMA
42       user_version` migration ladder in `backend/infrastructure/database.py`.
43
44     ## Committed guardrails (non-negotiable)
45     - **Paid LLMs (Gemini/OpenAI/Anthropic) = inference / serving / fallback ONLY ? never a
46       training data source** (terms/copyright). Commercial-clean licensing for EVERY
dependency ?
47       code, data, AND weights: **MIT / Apache-2.0 / BSD only** (ratified 2026-06-09, owner-
agreed).
48     - **Owned-model base:** **Qwen3-VL (Apache-2.0)** primary, InternVL3 (MIT) fallback;
**Gemma-4
49       is AMBER / bench-only** (Apache grant likely but Prohibited-Use posture unconfirmed ?
counsel
50       needed; do NOT build on it without sign-off). **Reject Gemma 3** (viral license).

```



```

**Exclude
51     minors** from the training pool; per-user training data needs a separate explicit opt-
in.
52     - **v1 = rally + highlight + history + correction loop**; **gait/biometrics strictly
future**
53     (GDPR Art. 9).
54     - **Generic data schema**; `coach ? athlete` is an **optional overlay**, not baked in.
55
56     ## Workflow conventions
57     - **Branch off `master`; ONE PR per work item; NO PR stacking.** Don't open a PR unless
asked.
58     - Tests: `python run_all_tests.py` (offline, fully mocked). Python 3.8+; `ffmpeg`
required.
59     - Never push to the default branch directly; descriptive commit messages.
60     - ?? This dev container lacks GPU / torch / cv2 / numpy ? the **full pipeline can't run
here**.
61     Verify pure-logic paths and hand machine-side checks (real video, GPU/WSL suite) to
the owner.
62

```

44. assistant (2026-06-13T18:45:51.692Z)

Now let me check for tests related to experiment setup and config:

45. user (2026-06-13T18:45:52.360Z)

```

C:\Users\avidu\Projects\badminton-highlight-indexer\tests/test_calibrate_local.py:21:    assert
len(cl.local_grid()) == 5 * 3 * 3 * 4    # velocity x merge x min_dur x min_crossings
C:\Users\avidu\Projects\badminton-highlight-indexer\tests/test_experiment.py:58:    v =
Variant(name="gateA", min_crossings=2, cue_flags={"person": False})
C:\Users\avidu\Projects\badminton-highlight-indexer\tests/test_experiment.py:66:    v =
Variant.from_dict({"name": "x", "min_crossings": 1, "bogus": 99})
C:\Users\avidu\Projects\badminton-highlight-indexer\tests/test_experiment.py:67:    assert
v.name == "x" and v.min_crossings == 1
C:\Users\avidu\Projects\badminton-highlight-indexer\tests/test_experiment.py:82:
min_crossings=0, min_players=0)
C:\Users\avidu\Projects\badminton-highlight-indexer\tests/test_experiment.py:96:    gate_a =
Variant(name="gateA", min_crossings=2)
C:\Users\avidu\Projects\badminton-highlight-indexer\tests/test_experiment.py:113:    preds =
predict(Variant(name="gateA", min_crossings=2), vc)
C:\Users\avidu\Projects\badminton-highlight-indexer\tests/test_experiment.py:151:    res =
compare(videos, CONTROL, Variant(name="gateA", min_crossings=2), iou=0.5)
C:\Users\avidu\Projects\badminton-highlight-indexer\tests/test_experiment.py:169:    res =
compare_unlabeled(vc, CONTROL, Variant(name="gateA", min_crossings=2))
C:\Users\avidu\Projects\badminton-highlight-indexer\tests/test_experiment.py:202:    res =
compare([_held_shuttle_video()], CONTROL, Variant(name="gateA", min_crossings=2))
C:\Users\avidu\Projects\badminton-highlight-indexer\tests/test_export_wasb_segments.py:86:def
test_run_min_crossings_gates_low_crossing_segments(tmp_path):
C:\Users\avidu\Projects\badminton-highlight-indexer\tests/test_export_wasb_segments.py:89:    r0
= ex.run(traj, fps=10.0, frame_width=100.0, out_csv=str(tmp_path / "o0.csv"), min_crossings=0)
C:\Users\avidu\Projects\badminton-highlight-indexer\tests/test_export_wasb_segments.py:90:    r2
= ex.run(traj, fps=10.0, frame_width=100.0, out_csv=str(tmp_path / "o2.csv"), min_crossings=2)
C:\Users\avidu\Projects\badminton-highlight-indexer\tests/test_rally_gate.py:3:from
backend.pipeline.detectors import rally_gate as rg
C:\Users\avidu\Projects\badminton-highlight-indexer\tests/test_rally_gate.py:38:    gated =
rg.gate_windows(pts, windows, fps, min_crossings=2)
C:\Users\avidu\Projects\badminton-highlight-indexer\tests/test_rally_gate.py:41:    assert
rg.apply_gate(pts, windows, fps, min_crossings=2) == [(0.0, 1.0)]

```

46. assistant (2026-06-13T18:45:53.749Z)

Excellent! Now let me check the test_experiment.py to understand the Variant and control infrastructure:

47. user (2026-06-13T18:45:54.001Z)

```
1      """Tests for the experiment harness (backend.eval.experiment).
2
3      Pure/offline ? synthetic candidate rows, no GPU/WSL. The load-bearing test is the
4      **no-regression invariant** (`test_disabled_cues_byte_identical_to_control`): a variant
5      that merely carries a disabled cue must predict exactly what control predicts. That is
the
6      guarantee that lets new cues be merged freely (`EXPERIMENT_HARNESS.md` §6).
7      """
8      import json
9
10     import pytest
11
12     from backend.eval.classifier import LogisticRegression
13     from backend.eval.experiment import (
14         CONTROL,
15         ExperimentError,
16         Variant,
17         VideoCandidates,
18         compare,
19         compare_unlabeled,
20         evaluate_variant,
21         load_video_candidates,
22         predict,
23         record_experiment,
24     )
25     from backend.infrastructure.database import Database
26
27
28     # A video whose third candidate is a held-shuttle false fire (net_crossings=0): it does
29     # not overlap any golden rally, so it is a negative by construction.
30     def _held_shuttle_video(name="v", gts=None):
31         return VideoCandidates(
32             name=name,
33             intervals=[(0.0, 10.0), (20.0, 30.0), (12.0, 13.0)],
34             features={"net_crossings": [4.0, 3.0, 0.0]},
35             gts=gts if gts is not None else [(0.0, 10.0), (20.0, 30.0)],
36         )
37
38
39     # Three videos where feature "x" perfectly separates rallies (x=5) from junk (x=0),
40     # so an honest LOVO learned variant should recover F1?1.0 on each held-out video.
41     def _separable_videos():
42         vids = []
43         for k in range(3):
44             base = 100.0 * k
45             vids.append(VideoCandidates(
46                 name=f"sep{k}",
47                 intervals=[(base + 0, base + 10), (base + 20, base + 30),
48                     (base + 40, base + 41), (base + 50, base + 51)],
49                 features={"x": [5.0, 5.0, 0.0, 0.0]},
50                 gts=[(base + 0, base + 10), (base + 20, base + 30)],
51             ))
52         return vids
53
54
55     # ----- Variant
56
57     def test_variant_roundtrip_and_hash_stability():
58         v = Variant(name="gateA", min_crossings=2, cue_flags={"person": False})
59         assert Variant.from_dict(v.to_dict()) == v
60         assert v.spec_hash() == v.spec_hash() # stable
61         assert CONTROL.spec_hash() == Variant(name="control").spec_hash()
62         assert v.spec_hash() != CONTROL.spec_hash() # different recipe ?
```

```

different hash
63
64
65     def test_from_dict_ignores_unknown_keys():
66         v = Variant.from_dict({"name": "x", "min_crossings": 1, "bogus": 99})
67         assert v.name == "x" and v.min_crossings == 1
68
69
70     def test_is_learned():
71         assert not CONTROL.is_learned()
72         assert Variant(name="r", feature_names=["x"]).is_learned()
73         assert Variant(name="p", classifier_model="model.json").is_learned()
74
75
76     # ----- the no-regression invariant
77
78     def test_disabled_cues_byte_identical_to_control():
79         """A variant carrying only DISABLED cues must equal control, window-for-window."""
80         vc = _held_shuttle_video()
81         disabled = Variant(name="fusion_off", cue_flags={"person": False, "audio": False},
82                             min_crossings=0, min_players=0)
83         assert predict(disabled, vc) == predict(CONTROL, vc)
84
85
86     def test_control_is_merged_candidates():
87         vc = VideoCandidates(name="v", intervals=[(0.0, 10.0), (10.5, 12.0), (40.0, 50.0)])
88         # merge_gap=1.0 stitches the 0.5s gap between the first two; the 28s gap stays
89         split.
90         assert predict(CONTROL, vc) == [(0.0, 12.0), (40.0, 50.0)]
91
92     # ----- decision gates
93
94     def test_gate_a_drops_low_crossing_window():
95         vc = _held_shuttle_video()
96         gate_a = Variant(name="gateA", min_crossings=2)
97         preds = predict(gate_a, vc)
98         assert (12.0, 13.0) not in preds # held-shuttle fire
99         removed
100         assert preds == [(0.0, 10.0), (20.0, 30.0)]
101         changed output
102         assert preds != predict(CONTROL, vc) # the gate actually

```

48. user (2026-06-13T18:45:54.272Z)

```

1     """Experiment harness ? A/B + shadow + side-by-side for rally detection.
2
3     This is the thin **variant/experiment layer** that `docs/EXPERIMENT_HARNESS.md`
4     identifies as the one missing piece: ~80% of the machinery (scoring, LOVO, the
5     no-regression gate, the pinnable classifier artifact, the persisted-candidate
6     substrate) already exists ? this module composes it. **No new scoring math lives
7     here**; everything defers to:
8
9     - `backend.eval.metrics`      ? `score`, `match_intervals`, `interval_iou`
10    - `backend.eval.training`      ? `label_candidates` (y by the same IoU matcher)
11    - `backend.eval.classifier`    ? `LogisticRegression` (the pinnable JSON model)
12    - `backend.eval.gemini_refine` ? `merge_overlaps` (the blessed window merge)
13    - `backend.eval.regression`    ? `gt_hash` (label-set fingerprint)
14    - `backend.eval.calibration`   ? `pct_improvement`
15
16    The thesis (`EXPERIMENT_HARNESS.md` §2): separate the **expensive, risky DETECTION**
17    (per-frame signals ? run once on the GPU/WSL box, persisted into
18    `candidate_segments.stats`) from the **cheap, swappable DECISION** (given those
19    signals, where are the rallies). A `Variant` is a declarative re-scoring recipe;
20    given persisted candidate features it produces `List[Interval]` deterministically,

```

```

21 with no GPU and zero production risk. So most experiments are pure offline
22 re-scoring of stored data and never touch the served pipeline.
23
24 Three modes (`EXPERIMENT_HARNESS.md` §5):
25 - `compare()` ? labeled A/B vs golden labels: per-video + LOVO-honest
26   aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-
27   held-out loop, but variant-parameterised.
28 - `compare_unlabeled()` ? SxS on NEW footage with no ground truth: where do two
29   variants agree / diverge (A-only, B-only, agreed). The divergences are what a
30   human spot-checks (and what two highlight reels would show side by side).
31 - the promotion gate stays `run_regression.py` + `regression_baseline.json`
32   (exit 0/1/3); **rollback = flip the variant/config, never `git revert`.**
33
34 Pure + offline + unit-tested. The only DB-touching helper is
35 `load_video_candidates`, which reads persisted candidates via
36 `Database.query_candidate_segments`.
37 """
38 from __future__ import annotations
39
40 import json
41 import logging
42 import os
43 from dataclasses import dataclass, field
44 from datetime import datetime, timezone
45 from hashlib import sha1
46 from typing import Any, Dict, List, Optional, Sequence
47
48 from backend.eval.calibration import pct_improvement
49 from backend.eval.classifier import LogisticRegression
50 from backend.eval.gemini_refine import merge_overlaps
51 from backend.eval.metrics import Interval, match_intervals, score
52 from backend.eval.regression import gt_hash
53 from backend.eval.training import label_candidates
54
55 logger = logging.getLogger(__name__)
56
57 # Where a comparison run appends one reproducible record. Tracked location (NOT under
58 # the gitignored output/) so the leaderboard survives a clean checkout, mirroring
59 # regression.DEFAULT_BASELINE_PATH.
60 DEFAULT_LEADERBOARD_PATH = os.path.join("eval_baselines", "experiment_leaderboard.json")
61 _METRICS = ("precision", "recall", "f1", "mean_iou")
62
63
64 class ExperimentError(ValueError):
65     """A variant cannot be evaluated as configured (e.g. a learned variant asked to
66     score an unlabeled video with no pinned model, or a gate referencing an absent
67     feature)."""
68
69
70 # ----- Variant
71
72
73 @dataclass
74 class Variant:
75     """A declarative re-scoring recipe ? NOT a code branch (`EXPERIMENT_HARNESS.md` §4).
76
77     A variant turns one video's persisted candidate windows + features into rally
78     intervals. Every knob defaults to the **control** behaviour (no gate, no learned
79     filter), so a variant that merely *carries* a new, disabled cue is byte-identical
80     to control ? the core no-regression guarantee.
81
82     Fields:
83     - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ? informational provenance
84       for the leaderboard and for selecting the served path (the existing
85       `segmenter_registry` + `IndexingConfig` do the actual selection in production).

```

```

86         New cues are recorded here default-OFF.
87     - ``min_crossings`` ? decision-gate **Gate A**: keep only windows whose persisted
88       ``net_crossings`` feature is >= this. 0 = off. This is the eval-only gate from
89       ``rally_gate.py`` expressed as a re-scoring knob (kills service-feed / held-
shuttle
90       windows ? see ``MULTI_SIGNAL_FUSION_PLAN.md`` §3.1).
91     - ``min_players`` ? decision-gate **Gate B** (the future person cue): keep windows
92       whose persisted ``players_in_court`` is >= this. 0 = off (no-op until the person
93       cue persists that feature).
94     - ``classifier_model`` ? path to a frozen ``LogisticRegression`` JSON. When set,
95       ``compare()`` scores videos with the SHIPPED model as-is (no per-fold training);
96       required for ``compare_unlabeled`` on a learned variant.
97     - ``feature_names`` ? the persisted features the learned filter consumes. When set
98       WITHOUT ``classifier_model``, ``compare()`` trains a fresh model per LOVO fold
99       (honest) ? the recipe, not a pinned artifact.
100    - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
101      overlap-merge gap (seconds), the same ``gemini_refine.merge_overlaps`` default.
102    """
103
104    name: str
105    segmenter: str = "wasb_hybrid"
106    config_overrides: Dict[str, Any] = field(default_factory=dict)
107    cue_flags: Dict[str, bool] = field(default_factory=dict)
108    min_crossings: int = 0
109    min_players: int = 0
110    classifier_model: Optional[str] = None
111    feature_names: Optional[List[str]] = None
112    threshold: float = 0.5
113    merge_gap: float = 1.0
114
115    def is_learned(self) -> bool:
116        """True if this variant applies a learned classifier (pinned model or
recipe)."""
117        return self.classifier_model is not None or bool(self.feature_names)
118
119    def to_dict(self) -> dict:
120        return {
121            "name": self.name,
122            "segmenter": self.segmenter,
123            "config_overrides": self.config_overrides,
124            "cue_flags": self.cue_flags,
125            "min_crossings": self.min_crossings,
126            "min_players": self.min_players,
127            "classifier_model": self.classifier_model,
128            "feature_names": self.feature_names,
129            "threshold": self.threshold,
130            "merge_gap": self.merge_gap,
131        }
132
133    @classmethod
134    def from_dict(cls, d: dict) -> "Variant":
135        known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]
136        return cls(**{k: v for k, v in d.items() if k in known})
137
138    def spec_hash(self) -> str:
139        """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard
140        and makes a result reproducible. The pinned **control** variant always hashes
the
141        same, so a regression is always traceable to a recipe change, never to drift."""
142        blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
143        return sha1(blob.encode()).hexdigest()[:12]
144
145
146    #: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
147    #: filter. Always the comparison reference; "rollback" = make this the served variant.

```

```
148     CONTROL = Variant(name="control")
149
150
```

49. assistant (2026-06-13T18:45:55.953Z)

Perfect! Now I have a complete picture. Let me synthesize this into a comprehensive, precise report on the no-regression risks and test plan.

50. user (2026-06-13T18:47:23.780Z)

Output does not match required schema: root: must have required property 'findings', root: must have required property 'anchors'

51. user (2026-06-13T18:48:59.977Z)

Structured output provided successfully

52. assistant (2026-06-13T18:49:16.777Z)

Perfect. Now let me provide a summary response to the user.

Summary: No-Regression Risks & Pure-Logic Test Plan for FusionSegmenter + Gate A

I have enumerated the complete no-regression test surface and the pure-logic (offline, zero-GPU) proof plan for the FusionSegmenter wiring with Gate A (min_crossings). The analysis is grounded in **actual file:line citations** from your codebase.

Core Regression Contract (Immutable)

wasb_hybrid and tracknet_hybrid must output byte-identical results when:

- `min_crossings=0`` (Gate A disabled, default)
- All `cue_flags={"person": False, "audio": False, ...}`` (all fusion cues disabled)

This is enforced by `test_disabled_cues_byte_identical_to_control`` (test\_experiment.py:78), which already exists and will be your regression gate. FusionSegmenter must **not modify** `TrajectoryHybridSegmenter.process_video()`` logic; it consumes the output only.

10 Specific Regression Risks (with line anchors)

1. **min_crossings in IndexingConfig** ? must default to 0, not 2
(backend/config/models.py:78?127)
2. **Gate A applied too early** ? FusionSegmenter must apply gate_windows() POST-Phase-2 (after candidate enumeration), not during it
3. **PersonDetector unconditional** ? must skip instantiation when cue_flags["person"]=False
(backend/pipeline/detectors/person_detector.py:56)
4. **In-place mutation** ? FusionSegmenter must not mutate `all_candidate_windows`` dicts; use copies
5. **Player-count gate references absent feature** ? min_players gates must default to 0; players_in_court feature is P3, not P2
6. **Court-mask applied too early** ? filtering during Phase 2 changes window enumeration; apply post-Phase-2
7. **net_crossings in stats** ? crossing count is a feature for distill_local.py:FEATURE_NAMES, not a stat
8. **Config override to high min_crossings** ? existing videos run with unexpected gating
9. **Trajectory hybrid equivalence broken** ? any logic change in base affects both wasb_hybrid and tracknet_hybrid
10. **Variant spec_hash drift** ? min_crossings, min_players must be Variant dataclass fields (already are, backend/eval/experiment.py:108?109)

Pure-Logic Test Plan (6 test suites, all offline/mock)

```

**Suite 1: Config + Equivalence (reuse existing)**
- Extend test_config.py to verify min_crossings=0 default + override safety
- test_experiment.py::test_disabled_cues_byte_identical_to_control ? **the load-bearing regression gate**
- test_experiment.py::test_gate_a_drops_low_crossing_window ? held-shuttle false fire drops when min_crossings=2

**Suite 2: Rally Gate Logic (extend test_rally_gate.py, mostly reuse)**
- test_gate_with_min_crossings_0_is_no_op ? min_crossings=0 filters nothing
- test_crossing_count_stability_across_fps ? crossing count is fps-invariant

**Suite 3: PersonDetector Window Enrichment (reuse test_person_detector.py idiom)**
- test_person_windows_with_cue_disabled_not_computed ? PersonDetector skipped when cue_flags["person"]=False
- test_person_detector_returns_window_dict_with_motion_stats ? windows have {peak_velocity, mean_velocity, track_id_count}

**Suite 4: Fusion Gate Composition (new)**
- test_court_mask_polygon_filters_persons_outside_court
- test_player_count_constraint_expects_2_or_4
- test_gate_a_and_gate_b_composition_both_must_pass ? AND logic: both gates must pass for window to survive
- test_default_off_gates_all_inclusive ? min_crossings=0, min_players=0 ? control

**Suite 5: Synthetic Fixtures (new, precise)**
- _held_shuttle_video with net_crossings=[4, 3, 0] (held window has 0 crossings)
- Trajectory points (rally with 4+ crossings, toss with 1)
- Person detections (two-player rally vs single-player feed)
- Court polygon (inside/outside spectators)
- Mixed feature fixture for AND-gate composition proof

**Suite 6: Byte-Identity Assertions (new, critical)**
- test_fusion_segmenter_with_all_cues_disabled_matches_trajectory_hybrid ? output row-by-row identical
- test_persisted_candidate_stats_unchanged_with_fusion_off ? stats dict keys unchanged
- test_handoff_window_boundaries_stable_with_min_crossings_0 ? Phase 3 clips use unfiltered boundaries

```

Owner-Verified GPU Tests (Real Video ? 5 cases)

1. ****Held-shuttle false-positive fix**** ? Gate A (min_crossings?2) drops the single-toss service-feed window
2. ****Gate B player-count rejection**** ? Solo warmup dropped when min_players=2 (future)
3. ****Two-sided motion**** ? Rally has both-sides motion; feed has one-side only
4. ****Full-video byte-identity**** ? wasb_hybrid(video) ? FusionSegmenter(video, min_crossings=0) on 10-min clip
5. ****Performance overhead**** ? Gate A = negligible; person cue (if enabled) = <10% additional wall time

All fixtures, test code, and line references are in the StructuredOutput above. Use test_experiment.py::Variant and the existing `predict()` function to drive most logic tests offline.